Final Project Report

Student Name: A Student

Student Number: 1234567

Location/Site: City Campus

Module Code: 6CS007

Module Name: Project and Professionalism

Project Title: Strategies for Teaching Deep Learning AI to play Video Games

Supervisor Name: A Lecturer

Reader Name: Another Lecturer

Submission Date: Today

Award Title: BSc (Hons) Computer Science

Declaration Sheet

Presented in partial fulfilment of the assessment requirements for the above award.

This work or any part thereof has not previously been presented in any form to the University or to any other institutional body whether for assessment or for other purposes. Save for any express acknowledgements, references and/or bibliographies cited in the work. I confirm that the intellectual contents of the work is the result of my own efforts and of no other person.

It is acknowledged that the author of any project work shall own the copyright. However, by submitting such copyright work for assessment, the author grants to the University a perpetual royalty-free licence to do all or any of those things referred to in section 16(i) of the Copyright Designs and Patents Act 1988. (viz: to copy work; to issue copies to the public; to perform or show or play the work in public; to broadcast the work or to make an adaptation of the work).

Student Name (Print): A Student

Student Number: 1234567

Signature: …….…… Date: Today

(Must include the unedited statement above. Sign and date)

Please use an electronic signature (scan and insert)

Table of Contents

1 Chapter 1: Introduction 8

1.1 Project Motivation 8

1.2 Introduction to Deep and Machine Learning 9

1.3 Deep Learning and Different Approaches to Machine Learning 9

1.4 Aims and Objectives 10

1.4.1 Academic Question 10

1.4.2 Aims 11

1.4.3 Objectives 11

1.5 Scope and Limitations 11

1.6 Report Framework 12

1.7 Summary 13

2 Literature Review of Machine Learning 14

2.1 Introduction 14

2.2 Neural Networks in Video Games 14

2.3 Different Methods of Teaching 16

2.4 Application of Neural Networks 19

2.5 Machine Learning Frameworks 20

2.6 Processing Power 21

2.7 Summary 22

3 Implementation and Development 24

3.1 Introduction 24

3.2 Design 24

3.2.1 Machine Learning Framework 24

3.2.2 Video Games Program 24

3.2.3 Structure of Network 25

3.3 Implementation 27

3.3.1 Decisions on Developmental Tools 27
3.3.2 Structure of Neural Network 27
3.3.3 Development 29
3.3.4 GPU Acceleration 30
3.3.5 Tools 30
3.3.6 Performance and RAM 31
3.4 Summary 31
4 Investigation and Comparison of Machine Learning Techniques 33
4.1 Experiment 1 – Initial Control, Ms. Pac-Man 33
4.1.1 Description 33
4.1.2 Challenges 35
4.1.3 Evaluation and Results 36
4.1.4 Conclusion 37
4.2 Experiment 2 – Reduced Large Rewards 37
4.2.1 Description 37
4.2.2 Challenges 38
4.2.3 Evaluation and Results 38
4.2.4 Conclusion 40
4.3 Experiment – No Large Rewards 40
4.3.1 Description 40
4.3.2 Challenges 40
4.3.3 Evaluation and Results 41
4.3.4 Conclusion 42
4.4 Experiment – Negative Reward for Death 43
4.4.1 Description 43
4.4.2 Challenges 43
4.4.3 Evaluation and Results 43

4.4.4 Conclusion 45
4.5 Experiment – Reward for Time Alive 45
4.5.1 Description 45
4.5.2 Challenges 45
4.5.3 Evaluation and Results 46
4.5.4 Conclusion 47
4.6 Experiment - Colour 47
4.6.1 Description 47
4.6.2 Challenges 47
4.6.3 Evaluation and Results 49
4.6.4 Conclusion 50
4.7 Experiment – High Epsilon 50
4.7.1 Description 50
4.7.2 Challenges 51
4.7.3 Evaluation and Results 51
4.7.4 Conclusion 52
4.8 Experiment – Breakout Control 52
4.8.1 Description 52
4.8.2 Challenges 53
4.8.3 Evaluation and Results 54
4.8.4 Conclusion 55
4.9 Experiment – Breakout Layering 55
4.9.1 Description 55
4.9.2 Challenges 55
4.9.3 Evaluation and Results 56
4.9.4 Conclusion 57
4.10 Experiment – Ms. Pac-Man Layering 57

4.10.1 Description 57
4.10.2 Challenges 58
4.10.3 Evaluation and Results 59
4.10.4 Conclusion 59
4.11 Experiment – Other Games 60
4.11.1 Description 60
4.11.2 Challenges 60
4.11.3 Evaluation and Results 60
4.11.4 Conclusion 69
4.12 Summary 69
5 Overall Results and Discussion 71
5.1 Introduction 71
5.2 Challenges 71
5.3 Strategies 72
5.4 Summary 73
6 Conclusion 74
6.1 Aims and Objectives 74
6.2 Overall 74
7 Critical Evaluation and Processes 77
7.1 Overall 77
7.2 Design 77
7.2.1 Positives 77
7.2.2 Areas for Improvement 77
7.3 Implementation 77
7.3.1 Positives 77
7.3.2 Areas for Improvement 78
7.4 Experimentation 78

7.4.1 Positives 78
7.4.2 Areas for Improvement 78
7.5 Testing and Errors 78
7.5.1 Positives 78
7.5.2 Areas for Improvement 79
7.6 Time Management and Planning 79
7.6.1 Positive 79
7.6.2 Areas for Improvement 80
7.7 Research and Learning 80
7.7.1 Positives 80
7.7.2 Areas for Improvement 81
7.8 Reflection 81
References 83
Bibliography 88
Appendices 90
Appendix A1 - Initial Gantt Chart 90
Appendix A2 - Final Gantt Chart 93

1 Chapter 1: Introduction

1.1 Project Motivation

Currently, deep learning artificial intelligence is a prominent subject in the technology industry. This project seeks to understand the challenges and strategies used in teaching deep learning AI's and to research the most efficient techniques to do so.

Deep learning is a subset of machine learning that fulfils certain conditions. These are the use of multiple layers in a neural network to compute a result, the ability to classify patterns in data through supervised or unsupervised training, and the ability to analyse different levels of abstraction of that data (LeCun, Bengio and Hinton, 2015).

The problem presented to the AI will be video-games, initially the game Ms. Pac-Man. Ms. Pac-Man was originally an arcade game, where the objective is to guide Ms. Pac-Man into eating all the pellets in a maze while avoiding the ghosts that traverse it. As the ghosts can end the game if they touch the player, and their movement is non-deterministic, this takes a certain level of observation and reaction from a human player (Williams, Perez-Liebana and Lucas, 2016).

A Machine Learning framework will be integrated with a version of Ms. Pac-Man and then trained to play the game, taking the place of the human player. The framework will be tested using different techniques, such as different heuristics, rewards, and game state information being passed to the network, as well as changes to the structure of the network, and the advantages and disadvantages of each technique are compared and evaluated.

The goal of the project is not just for an AI to be able to play Ms. Pac-Man, as has been achieved in previous research, but to investigate the methods used and to determine the most effective ways to train the machine learning model. For example, one aspect of training is the heuristics used to qualify "success" (e.g. score, time alive, total distance moved), which can change how the AI learns and then behaves in the game.

For further research, these techniques could then be applied to other games. For example, testing the same network and parameters on different game environments (e.g. Pong, Space Invaders, etc.) could give very different learning rates and

behaviours. These can then be compared and used to determine what kind of games are able to be trained most effectively using deep learning.

Therefore, the question of " What are the challenges and most efficient strategies for teaching machine learning AI to play video games?", is to be researched by investigating these various aspects of machine learning, training the AI with several methods, and then evaluating the performances of the outcomes.

1.2 Introduction to Deep and Machine Learning

Deep learning is based on the use of neural networks, an artificial representation of the neurons in a biological brain (Hagan, Demuth and Beale, 1996). Neurons receive an input and calculate an output based on a mathematical function. By passing information through these neurons, observing the results, and then updating the function inside the neuron, the neurons can be trained to produce desirable outputs based on the input data given. This training process is referred to as machine learning, as the software is able to improve itself without explicit programming (Samuel, 1959).

Outputs from a set of neurons can be passed to another set of neurons as input, with each set known as a layer. Many neurons interconnected form a neural network, and a neural network with many layers is known as a deep neural network.

Deep learning requires a deep neural network, which is a neural network with many "hidden" layers, layers of neurons between the initial input layer and the final output layer. For this report "Deep Learning" and "Machine Learning" will be used interchangeably, due to the vaguer definition of deep learning, and the prominent use of deep neural networks in machine learning.

1.3 Deep Learning and Different Approaches to Machine Learning

Deep learning is a specific type of machine learning, that uses many layers in a neural network for pattern analysis or classification of data (Deng and Yu, 2014). This differs from a traditional neural network by having more "hidden" layers, layers of neurons that are between the input and output layers that allow for more calculations.

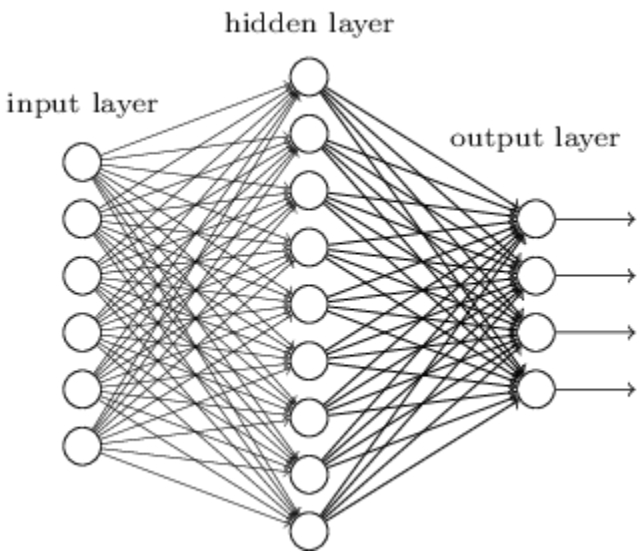


Figure 1.1 A typical neural network. (Nielsen, 2015)

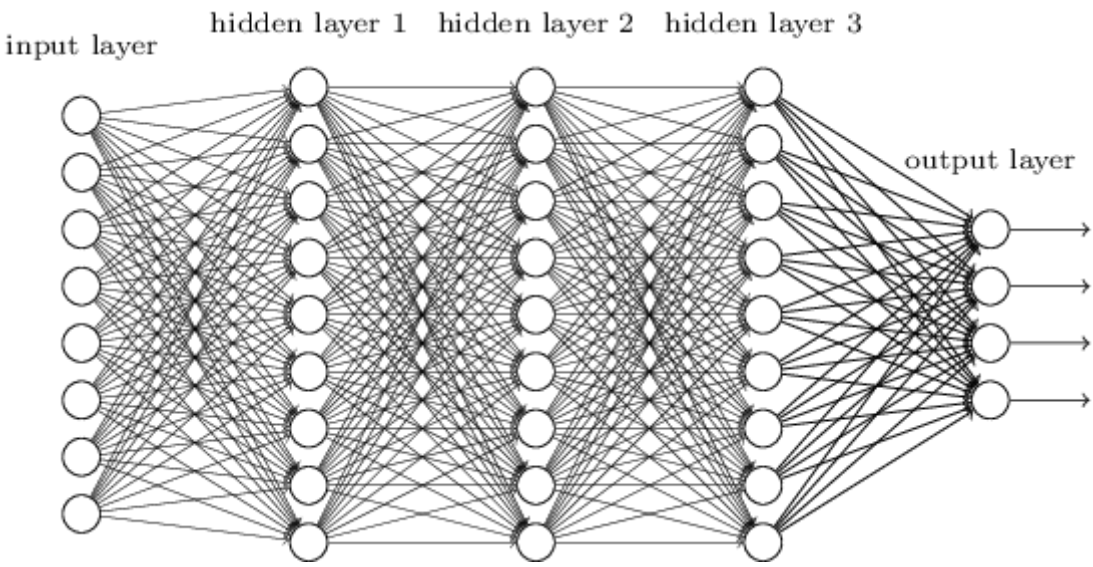


Figure 1.2 A 'deep' neural network, consisting of more hidden layers. (Nielsen, 2015)

Figure 1.1 shows a typical neural network with an input and output layer. Figure 1.2 shows a deep neural network consisting of an input and output layer, and also three hidden layers. These more numerous layers allow for more complex calculations between the layers, theoretically leading to more desired output. For this project, a deep learning neural network will be implemented and used to play a video game, with the game's data as the input, and the actions to take in the game as the output.

1.4 Aims and Objectives

1.4.1 Academic Question

The academic question of this report will be the following:

What are the challenges and most efficient strategies for teaching machine learning AI to play video games?

This question will be answered using the following aim and objectives, to thoroughly explore all aspects of machine learning usage in video games.

1.4.2 Aims

- What are the current strategies for teaching machine learning AI systems?
- What are the technologies and techniques used for making AI learn to play video games?
- Develop an AI framework to be able to play a commercial video game.

1.4.3 Objectives

- Research current strategies for machine learning AI
- Research current literature regarding machine learning AI playing video games
- Decide on a framework for machine learning (e.g. TensorFlow, scikit-learn)
- Choose implementation of Ms. Pac-Man game.
- Integrate the AI framework into the game, and have it able to take the place of a human player
- Train machine-learning AI to play Ms. Pac-Man and assess the efficiency of different learning methods
- Explore different strategies for training the AI, and compare advantages and disadvantages
- Choose and implement additional video games into the framework.
- Test the ability for the network to play additional video games and evaluate the network's effectiveness with them.

1.5 Scope and Limitations

The scope of the project will be initially limited to the programming of a deep learning neural network and then training it to play a single video game. This is to establish a baseline for future experiments and to determine if there are any challenges or limitations with expanding the scope of the project to accommodate more learning environments.

Predicted initial limitations included time and computing resources. Training time for neural networks can potentially take days depending on the resources available, size

of the network, and size of the data input. Therefore, it is important to perform an initial experiment to determine whether this will be feasible in the projects time scope.

Linked to this is a potential lack of computing resources. Due to the nature of machine learning, the network can be trained with little human intervention. However, it requires significant processing power to do so and a continuous uptime. Finding a machine that is consistently available with the necessary processing power could be a potential issue with the project.

1.6 Report Framework

This report consists of this seven chapters, consisting of an introduction to the topic, a literature review of existing machine learning AI, details on implementation and development, investigation of various experiments, overall results, conclusions, and finally a critical evaluation of the whole project.

Chapter 1 will introduce the topic of deep learning and its use in video games, as well as the overall structure and goal of the project.

Chapter 2 will be a literature review examining existing literature and academic research surrounding deep learning being used to play video games, to determine potential techniques and challenges that could occur while implementing a deep learning AI.

Chapter 3 will be the implementation and development section and will detail how the framework for the experiments was developed, and how it was used to perform experiments, as well as any modifications that needed to be made to complete the objectives of the project.

Chapter 4 will be a listing of each experiment and will contain details of the experiment setup, running, and then an analysis of the results. This will then be compared to previous experiments and will be used to give a general direction the project should take.

Chapter 5 will have a discussion on the overall results of the experiments, summarise findings, and have an initial comparison of all the experiments, parameters changed, and techniques used.

Chapter 6 will be an overall conclusion and will compare all experiments, techniques, and challenges, and discuss how this affected the project, and the overall goal of finding the most effective techniques and challenges of getting a deep learning AI to play video games.

Chapter 7 will be a critical evaluation of the project and its processes. This will determine which aspects of the project could have been improved, which aspects were done well, and what modifications were made to the project as it went on.

1.7 Summary

This chapter presented an introduction to the topic of deep learning, and its use in learning to play video games, as well as a definition of deep learning for the projects, and a summary of the structure of neural networks based on their usage in machine learning. Deep learning itself is a wide topic and restricting it to the problem of playing video games allows a more defined structure to evaluate the best methods and techniques

The academic question, aims, and objective for the project were defined and will be used to define the structure of the project, and the tasks required to fully complete it and answer the academic question.

Chapter 2 will be the literature review process that will give an overall view of existing research on the topic and highlight any techniques to use or potential pitfalls to avoid in the implementation of the network.

2 Literature Review of Machine Learning

2.1 Introduction

This chapter will be a literature review of surrounding research based on the subject of deep learning, neural networks, and their use in playing video games. This will compare, contrast, and evaluate different sources of research to provide a base of information from which to build the project on and to highlight potentially useful approaches and potential pitfalls.

2.2 Neural Networks in Video Games

Creating an AI to play video games is not a new idea, and the idea of using neural networks to play video games, specifically Pac-man, has been written about from at least 2001 (Kalyanpur and Simon, 2001). This paper also includes the use of "genetic algorithms" that mutate and combine algorithms to help prevent falling into a "local optimum", a state where a regular machine learning AI cannot improve the algorithm. Though this approach is useful, deep learning has the ability to overcome these local minima with the use of proper optimizing algorithms (Kawaguchi, 2016), and so will likely need to be studied further in the project to avoid these problems.

For the likely level of intelligence of the AI that can be feasibly trained, one paper utilising a neural network was able to train an AI to the level of a human novice, capable of clearing the first level, but also making frequent mistakes (Lucas, 2005). This is in contrast to the team from "Maluuba", who were able to produce an AI system that was able to achieve a score of "999,990" and beyond (van Seijen et al., 2017).

For that team, the problem was divided into smaller sub-problems, with an agent given to each smaller problem, and each working together to achieve overall goals. While this score is proven to be possible, it is unlikely to be the outcome of this project, due to the computing resources, the time and data needed (800 million frames of game data), and differing goals. A more likely outcome will be an agent that can play games such as Ms. Pac-Man to a novice human level, with different levels between different techniques. However, the technique of dividing up each problem into smaller problems and assigning them their own agents may be useful in comparing training techniques.

Another view of this multiple agent problem is a paper on modular neural networks (Schrum and Miikkulainen, 2014). Rather than a single "monolithic network", the game

was divided into the conceptual aspects of chasing edible ghosts and escaping from them at all other times. This was found to outperform singular neural networks and seemingly points towards a more effective technique by using multiple neural networks for different aspects of the game but introducing more complexity due to the communication between networks, and the additional processing power required.

For an example of machine learning in different game environments, a simple neural network can be created to beat pong, via a python script and the use of OpenAI gym (Parthasarathy, 2016). In terms of training time, it took 3-4 days before it was able to beat the game's built-in AI. Here the main aspects of the network were image pre-processing and then using a convolutional neural network. While was this took quite a while for training, this could have been sped up, likely by using a faster processor than a laptop, or as discussed later, by utilising a GPU.

As a non-playing example with video-games, one paper instead used a neural network to attempt to predict the next frames of a video game, based on the control input, and previous frame data (Oh et al., 2015). While this is not specifically useful for playing video games, it shows that models can be created to accurately predict the next frame or action of a video game accurately, based on information given to the game. The implications of this are that, in previous examples above, the networks only made decisions based on the data it had, and previous game data. This would enable the addition of future data to the model, to more accurately associate actions and future states of the game, potentially leading to more effective models. However, this would also depend on the frame-predicting network having been trained previously, negating some of its use, depending on how long that would take.

Deep learning more recently has been used to tackle a wide range of video games from Ms. Pac-Man and arcade games to real-time strategy and first-person shooter games. One paper covers the various implementations of deep learning applied to different video games genres (Justesen et al., 2017). While many games have different genres have been successfully trained, the paper also brings up challenges that have yet to be completely answered. Some relevant ones to the project include the current lack of a general game playing AI, as a network is usually set up and configured to train on a specific game.

Another problem is games that have very sparse rewards, such as "Montezuma's Revenge", where the rewards given by the game do not easily correlate to any specific action. The requirement for the agent to set their own goals, or manually implementing them is mentioned as a workaround for this particular issue. The paper also mentions the computational power being an issue, especially where multiple agents are required, such as the modular approaches mentioned (Schrum and Miikkulainen, 2014). A solution mentioned would be the use of smaller networks, but the main improvement would have to come from increased neural network processing power.

The paper also mentioned the lack of "lifetime learning" as a problem with current deep learning AI. With current deep learning solutions, for the AI to adapt to a new environment, or a change in the behaviour of the game, it would require almost a full retrain of the network from scratch. This causes problems with attempting to take a model and retraining it on a new scenario, especially when trying to adapt to a playing against a single human player as an example.

From these papers, it can be seen that deep learning in video games is a fairly broad topic and covers almost every genre of video game. For this project, either taking changing the environment the game works in or changing the game entirely will be useful to see how different techniques can be applied to different learning environments for the AI, with the potential to increase the scope wide through other genres of games or introducing playing data from a human. The next section will cover the techniques researched in teaching AI.

2.3 Different Methods of Teaching

Different approaches to the use of AI to play video games includes the use of route-trees, a precursor to machine learning algorithms (Robles and Lucas, 2009). This approach uses a screen-capture interface and hard-coded heuristic. On a simulator, the program scored 40,000, but on the original game using the screen-capture interface, only scored around 15,000. While not specifically using neural networks, an interesting idea here was the changing of heuristics. The highest scoring strategies were choosing a random safe path of the agent and following the path of pills in the game. The difference between those two methods shows that defining heuristics and rewards for the agent can have a dramatic effect on the performance of the AI.

A particularly effective way of training a network to play a video game was demonstrated using Atari games and reinforcement learning (Mnih et al., 2013). Here, Q Learning was combined with a deep neural network to analyse Atari games and learn from the actions, estimating the quality of the actions depending on the state of the game, and using them to determine future actions quality. Another interesting point was using raw screen data captured from the Atari games, which were 210x160 pixels, each with a 128-colour palette. This would result in a large amount of information needing to be processed, so the image was first converted to greyscale, and then scaling it down to a 110x84 image, and then cropping to only the relevant game area. This cropping was done due to their use of convolutional neural networks which expected a square input but could have been modified to use any shape input. The network then successfully learnt to play and approached state-of-the-art results in six of seven games played.

An example of changing parameters to affect game learning can be seen with a simple Snake machine learning game (Korolev, 2017). Here, the main parameters changed were the rewards given, and the overall goal for the AI. In the first example, the Snake was only given the goal of surviving and not running into itself. After learning, the AI player simply moved in a circle, not gaining points, and but never losing the game. When given the goal of gaining points, the snake was able to gain them, but quickly died by running into itself. By giving the network rewards for gaining score, and then also giving information allowing the snake to avoid itself, the game improved over both attempts. This that changing the heuristics and parameters of the network can drastically affect the results, which will likely form the basis for some of the experiments.

Outside of video games, a general paper on training deep neural networks looks at some of the problems encountered with training deeper networks (Glorot and Bengio, 2010). It shows that classical methods of training neural networks caused some networks to converge slower and to reach local minima, stunting training. Part of their conclusions was also the lack of explanation for many of their observations, meaning that the reasons some networks performed better than other for no reason that could be easily explained. This implies that changing the structure of the network can have unknown effects on the actual learning rate of the network and is likely to require experimentation during the project.

For general deep learning networks, this structure is an important factor, but another important factor is the algorithms used to train the network. One paper examined the effect of batch normalisation to accelerate deep learning (Ioffe and Szegedy, 2015). Here, the paper states that regular deep learning networks often require a lower learning rate, and very specific parameters to achieve a good result. Instead, the paper presents the idea of normalising the inputs between layers in neural networks. By doing this, it helps prevent small changes to the data amplifying throughout the network, and instead keeps it constrained based on the normalised values. This also means that the learning rate is able to be increased, leading to the faster training of the network, with an example of 14 times faster training time.

One of the more prominent examples of a successfully trained neural network is the success of DeepMind's AlphaGo (Glorot and Bengio, 2010). Here, the AI was trained to such a level that it was able to beat the top human player of Go, and other Go playing software. This was achieved by a multi-step process, first with a supervised system that was trained and evaluated using expert moves.

The next iteration used a reinforcement learning approach and, when trained, was able to beat the previous iteration around 80% of the time. This was then combined with an estimator function which helped limit the search space of the network, which allowed it to focus on a smaller set of moves to make.

One of the major improvements was the analysis of the network and determining that it may have been overfitting by only being trained on step by step complete games, which was hampering the generality of the network. To combat this, a set of game data was generated that was otherwise not connected to each other and used to help prevent the network from overfitting. These eventually allowed the network to achieve a level of playing that beat other commercial Go playing software 100% of the time, and eventually the top human player.

While Go itself is not a "useful" problem to solve for other applications, the paper notes that the techniques used to produce this AI has applications beyond just this game. While some of the techniques were specifically done to help Go, the ideas behind them, preventing overfitting, and reinforcement learning can be applied to a wide variety of other situations.

2.4 Application of Neural Networks

While the project's goal is examining the techniques and challenges with training AI system, this research is also applicable to real-world situations. One of the more recent applications of deep learning is in self-driving cars. One paper examines the use of simulators to train these autonomous vehicles, and the advantages and disadvantages of doing so (Chen et al., 2015).

One of the main issues with training self-driving cars is that real-world training would be very resources intensive, and potentially cause damage to the car or the environment, due to the trial-and-error nature of learning. The paper specifically mentions reinforcement learning that has been used to play video games as an example of one structure of network that can be used to train a self-driving car.

Due to the resource constraints, training in a simulator first is an ideal testbed for self-driving cars, allowing a controlled environment and a faster training speed than real-life scenarios. Eventually, a car will have to be trained in a real-world environment outside of a simulator, but any efficiencies that can be gained from simulated training can help cut down training times, improve accuracy, and generally reduce costs.

The use of simulators is explored further in a paper examining its uses for scenarios such as medicine, image recognition, and network intrusion (Luke et al., 2005). These simulators offer a game-like environment with which a machine learning system can be trained, and then applied to real-world situations. While these simulations, like the car simulator, may not match the real world exactly, they can offer a good baseline estimate and are still useful for initially teaching a neural network.

The creation and use of these simulators have a wide range of uses, with one such system simulating a traffic environment, with reinforcement learning applied to traffic control (Wiering, 2000). As traffic systems can be difficult to hand-tune, a reinforcement learning model was trained on how to best manage traffic lights, with rewards based on estimated and actual car waiting times. Here, the RL network outperformed a number of different fixed methods depending on the number of cars saturating the network, showing that reinforcement learning can be applied do a considerably different situation than video games.

Another way this is used is in the field of robotics, with one paper examining the use of multi-agent systems, with an example of robot soccer (Stone and Veloso, 2000).

The progression of explicitly programmed teams to fully machine learning trained team is discussed and was made possible due to the processing and machine learning advances made in fields other than robot soccer. For this project, while real-world applications may not be able to be explored due to time and scope, the techniques and challenges discovered could likely be linked back to real-world applications.

2.5 Machine Learning Frameworks

A range of software frameworks are available to assist in developing and implementing a neural network for machine learning. A few of these are Caffe, Neon, TensorFlow, scikit-learn, and Torch. These mostly vary by the core language used, as well as the syntax, and support for extended features such as threading, and GPU acceleration. These frameworks help abstract the coding of a neural network, and in theory make it easier to read and understand, and also make it easier to code.

From a 2016 study of various frameworks, various advantages of each framework were highlighted, mainly around performance and features (Bahrampour et al., 2015). The main conclusions were a preference for Torch and Theano based on CPU performances as well as extensibility and third-party libraries. TensorFlow is given a mention as an extremely flexible network, with growing community support, but its performance was not as competitive to other networks.

Different approaches are the use of cross-compatible primitives that can be used across machine learning frameworks (Milutinovic et al., 2017). This approach is similar a to a compatibility layer between the code written, and the frameworks processing it. While enabling interoperability, it is not necessarily required if using only a single framework and would more likely be useful in collaborative development efforts. In addition, as this is only a proposal for an API, it would not be worth using for the project.

A second comparative study examined Theano, Torch, Caffe, TensorFlow, and Deeplearning4J (Kovalev, Kalinovsky and Kovalev, 2016). This study used the MNIST database of handwritten digits to test the performance of each network. It also measured the complexity of use in each of the frameworks, mainly by lines of code. Through their testing of accuracy, speed, and features, the top-ranked frameworks were Theano, TensorFlow, and Caffe. This was also the same ranking of relative complexity. Combined with the results of the first study, focusing on features, Theano

and TensorFlow are mentioned as comparatively performant and flexible frameworks and should be fully considered if a machine learning framework is used.

A general survey of neural networks also helps to make sense of the differences between them (Zacharias, Barz and Sonntag, 2018). This survey covers previously mentioned frameworks such as TensorFlow, Keras, and Caffe, but also other frameworks such as MXNet, CNTK, and Chainer. The main feature of this survey is the comparison of popularity calculated from "GitHub metrics", such as number of forks, issues, and contributions. Here TensorFlow is used as a baseline value of 100, with the second highest being Keras at a relative value of 46.1. This should be taken as a rough metric, however, as GitHub contribution does not equal actual usage, the simplicity of use, or examination of other features available. However, it may make a case for extensive documentation, community support and tutorials, and potential compatibility with other tools, as it is likely that they would support the most popular libraries.

With these factors in mind, most of this research points to a few of the more popular frameworks as being more performant, or easier to use as a framework, namely TensorFlow, Theano, Keras, and Caffe. Notably, Keras itself is a framework on top of a framework, that can sit on top of TensorFlow and Theano (Keras, 2018). In terms of a choice for the project in terms of general popularity, support, and features, it seems that these four frameworks would be worth exploring for further use.

2.6 Processing Power

Part of the project is likely to involve a machine learning framework and significant neural network training time. More recent advances in processing power such as leveraging GPU's for processing has enabled much more complex and deeper neural networks (Parvat et al., 2017). This paper compares several frameworks including TensorFlow, Torch, and Theano, on aspects such as modelling capabilities, community support, and documentation quality, which is a good starting point for choosing a framework. The use of GPU acceleration can increase the speed of a typical CPU based program by up to 150x (Guzhva, Dolenko and Persiantsev, 2009), which would assist with the unknown time required for the project training time. However, the added complexity of adding GPU acceleration may take longer than would be saved by implementing it.

To speed up training times further, it could also be possible to reduce the information that is given to the network, trimming parts that are superfluous or not relevant. An example of this would be cropping and scaling down images from the screen capture, as done in the experiment to get a deep learning AI to play Atari games (Mnih et al., 2013).

Some difficulty in obtaining hardware to run training is expected for the project, as a high-end graphics card would be needed to train in a reasonable amount of time. To mitigate this, there are a number of Infrastructure-As-A-Service (IaaS) services, which offer virtual machines for computing tasks (Zhang, Cheng and Boutaba, 2010). These services such as Google Compute Engine, and Amazon Web Services, offer high-performance virtual machines with customisable CPU and GPU options to allow for a wide range of hardware options (Google, 2018a)(Amazon Web Services, 2018). While these would allow for fast training on high-end hardware, it would add complexity to remotely configuring and managing virtual machines, and would also be prohibitive in terms of cost, with a standard 8-core CPU and GPU virtual machine from Google Cloud Computer costing about $400 a month in constant use (Google, 2018b), so while useful for research purposes, would likely not be suitable for an academic project.

Another method of speeding up neural network training would be the use of clusters of computers, managed with something like MPI. One paper looked at the feasibility of distributing TensorFlow training across multiple cores using MPI (Vishnu, Siegel and Daily, 2016). The results were encouraging, with certain networks gaining a relative speedup of 13.34 using 40 cores, relative to 1 core. In addition, there were very minimal changes needed in the TensorFlow code required to run it across multiple cores using MPI. While this could be useful for the project, the ability to gain large amounts of computers to use in a cluster may be difficult. A solution might be to use the cloud VM services mentioned above, however, it would still be unfeasible due to costs.

2.7 Summary

This chapter presented key aspects gleaned from the literature review that may be useful to know for the project. This includes looking into GPU acceleration to greatly speed up training times, mostly to help keep the project within specified timeframes,

as well as the structure of the neural networks. Prior research has also been done not just with neural networks but adjusting the heuristics and information available to them to create a better performing AI, which is the bases for answering the project's academic question.

Other approaches include using modular neural networks and splitting the "problem" of the game into multiple problems which agents can solve. For the implementation of the AI, almost all research used a deep learning framework of some kind, which useful when trying to implement the project artefact.

The level of performance of the AI is likely to not be above the level of a novice human player but will be enough to show the AI learning, putting it at a level beyond random play, and allowing it to be compared with other experiments. Avoiding the "local minima" problem will need to be solved through careful application of deep learning optimisation algorithms.

While the initial focus will be on using Ms. Pac-Man, other games are likely to have similar problems as expressed in research and should be able to be mitigated through similar methods mentioned.

This information will be used in the implementation of the artefact and through the experiments performed by it, leading to answering the question of which techniques are the most effective for teaching a deep learning AI to play video games.

The next section will be the implementation and development of the project artefact, to allow training a machine learning framework to play a video game.

3 Implementation and Development

3.1 Introduction

In this chapter, the implementation and design of the machine learning framework are discussed, as well as design decisions made about the project, and any challenged found and tackled during the implementation phase of the project.

3.2 Design

3.2.1 Machine Learning Framework

One of the main decisions when developing the initial framework for the experiments is whether to use a machine learning framework and if so, which one to use to implement the deep learning setup.

While a simple neural network can be programmed with very few lines of codes and be effective at learning (Spencer-Harper, 2015), more complex networks would require more structure and functions which have already been developed in mature frameworks, essentially duplicating effort.

The decision to use a framework is due to two factors. One was the unknown time constraints, as it is unclear how long it would take to train a network to a competent level, and second is due to the development of the network not being part of the academic question, with the application of the network is the most important part.

After a brief analysis, the most likely choices are either Google's TensorFlow or scikit-learn. Both these frameworks use python as a high-level language, and a faster C++ backend for the actual processing, and have documentation available for faster set up of neural networks.

3.2.2 Video Games Program

For the project, Ms. Pac-Man was chosen as the initial game to use for experimentation, due to its use in AI research (Handa and Isozaki, 2008) and also due to its familiarity as a video game, which would make assessing its behaviour easier to understand. A screenshot of Ms Pac-Man can be seen in Figure 3.1 below.

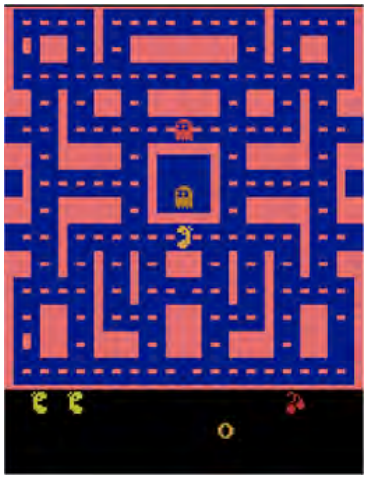


Figure 3.1 A screenshot of Ms Pac-Man running on an Atari 2600 emulator.

The specific implementation needs to be chosen on several factors, such as ease of modification, and ability to pass data outside the program. There are several implementations of Ms Pac-Man in a variety of languages, however, the most useful ones for this project would be coded in a familiar language such as Python, Java, JavaScript, or C++.

3.2.3 Structure of Network

A deep learning neural network can be configured in several ways. The simplest one is a Deep Feed Forward neural network.

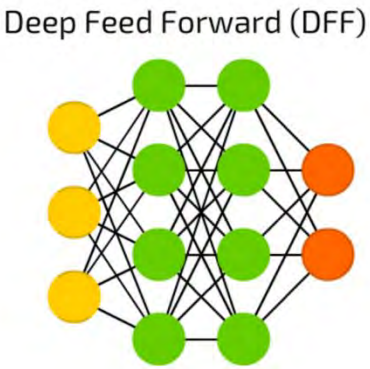


Figure 3.2. A Deep Feed Forward Neural Network Diagram. (Veen, 2016)

As shown in Figure 3.2, information travels in one direction, from input nodes (yellow) to output nodes (orange), with the weights of the nodes adjusted depending on if the output matches the expected output (Schmidhuber, 2015). The deep part of the network comes from the hidden layers, the layers between the input and output (green), which enables more complex behaviour in the network than what can be calculated from just a single layer (Auer, Burgsteiner and Maass, 2008).

Another type of network to consider is a Recurrent Neural Network. These networks have neurons that are given information from the previous layer, and themselves from a previous run.

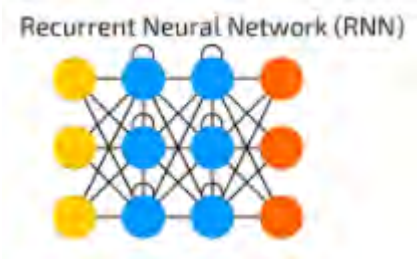


Figure 3.3. A diagram of a Recurrent Neural Network. (Veen, 2016)

As shown in Figure 3.3, the recurrent nodes (blue) are feeding information back into themselves. This gives recurrent neural networks a state that allows them to have a "memory" of previous actions (Li and Wu, 2014).

A third type of network to examine is a Deep Convolutional Network. This is a variant of a deep feedforward neural network that is mainly used to analyse visual imagery, such as handwriting (Ciregan, Meier and Schmidhuber, 2012).

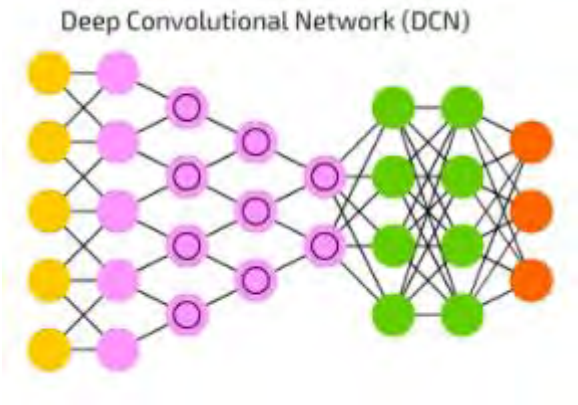


Figure 3.4. A diagram of a Depp Convolutional Network. (Veen, 2016)

This can be seen in Figure 3.4 using kernels (pink nodes) and convolutional nodes (pink with circles). These networks work most commonly by taking a section of an image (kernel) for example, feeding it through the network, and having each layer of convolutional node process smaller and smaller parts of the input. This then continues with another section of the image, continually until the complete image has been analysed (Ciregan, Meier and Schmidhuber, 2012).

The use of these networks will have to be decided based on the implementation of the game used for the artefact, and what kind of data is able to be extracted from the game.

3.3 Implementation

3.3.1 Decisions on Developmental Tools

The main deciding factor to which framework to use came down to documentation and community support available, with TensorFlow being seen as the most popular in terms of usage and community support (Bahrampour et al., 2015). A large amount of community support meant that setting up a neural network could be done more quickly, but also importantly be properly debugged for errors more easily. Its use in papers in the literature review also contributed, proving its effectiveness.

For the implementation of the game to be played, the use of an Atari emulator with the OpenAI Gym framework (OpenAI, 2018) was chosen, as it provided a standardised way of retrieving data from the games, providing input, as well as being able to switch the emulator to different games with a standard interface.

After deciding on TensorFlow, Python was chosen as the main developmental language, mainly due to its use as the main scripting language of TensorFlow and OpenAI gym, with the version Python 2 being used, to maintain the widest compatibility of potential modules that may be required for development.

Java would have been an acceptable alternative, however, the Java API for TensorFlow was not as stable or documented as the Python API (TensorFlow, 2018a), and so was not used due to the potential for unexpected behaviour.

3.3.2 Structure of Neural Network

The type of neural network was dependent mostly on the way data could be extracted from the game. For Ms. Pac-Man, OpenAI gym offered a standard emulator, which provided screen data, and a RAM version, which provided all 128 bytes of the emulator's RAM per frame.

A RAM version would most benefit from a Feedforward or Recurrent Neural Network, whereas the screen data would benefit from a Convolution Neural Network, due to the types of data being retrieved.

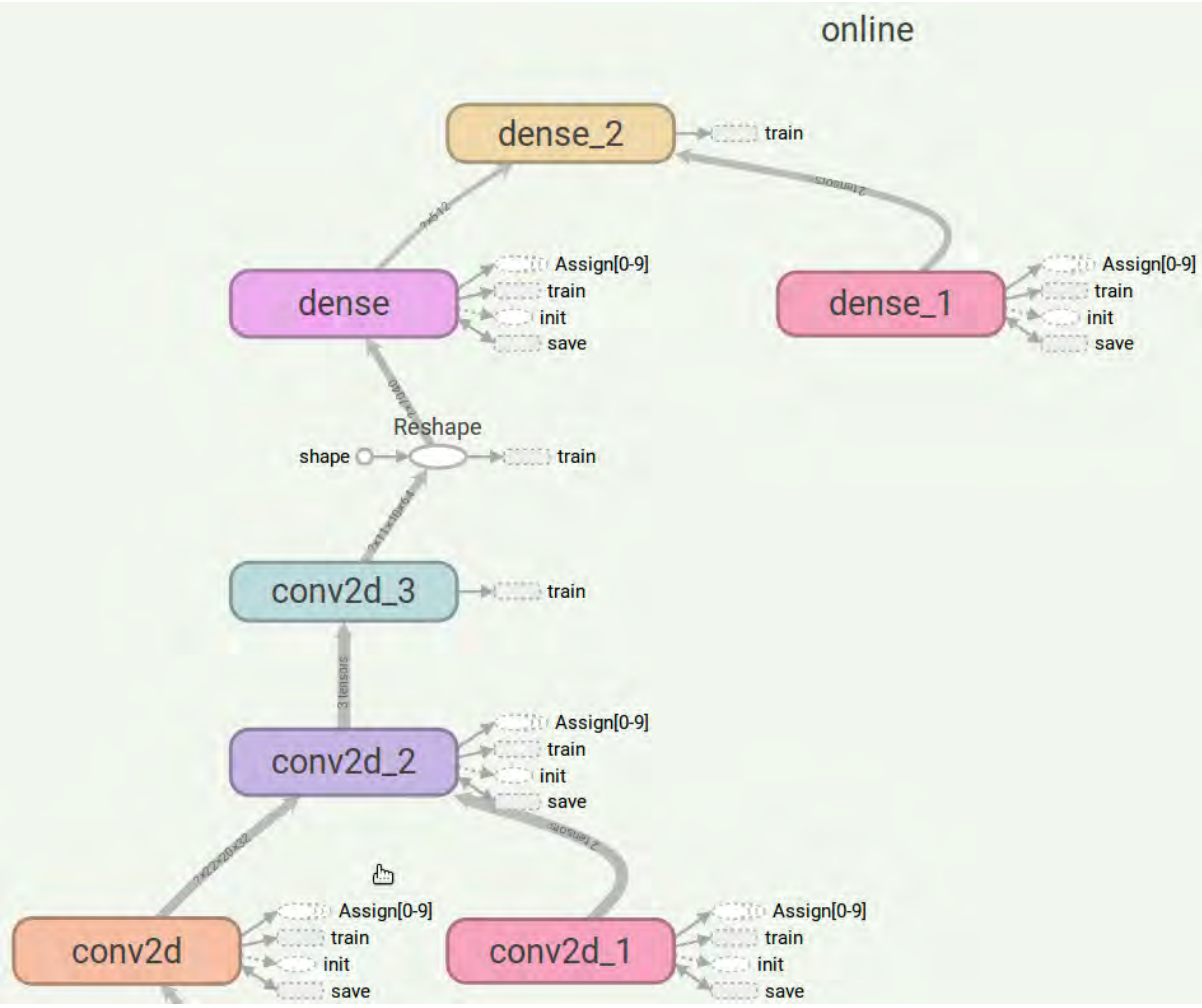


Figure 3.5 A generated diagram of the CNN used for the project, generated by TensorBoard.

In Figure 3.5, the structure of the network can be seen from a generated diagram using TensorBoard. This shows the 3 convolutional layers, followed by two dense layers, with input coming from the bottom.

For the artefact, the screen data version was used, mostly due to the ability to easily extract, process, and then display screen data, which would allow for the easier presentation of images from the game, especially if the images needed to be pre-processed before being analysed.

Due to the nature of the task being given to the network, a video game with possible actions and rewards based on the state of the game, a Q-Learning network was chosen as a reinforcement learning technique. Here the quality of the action is determined by trying an action in a certain state, evaluating the outcome of that state, and if it is desirable, increasing the q value of that action in that state, decreasing it for undesirable outcomes.

Due to the nature of Q-Learning, every step of training would adjust the values of the network, leading to different results at every iteration. While sometimes useful, it may lead to large amounts of a variance, and potentially creating a feedback loop where the network constantly performs the same action due to the predicted Q-value not changing sufficiently and leading to a stuck network (Juliani, 2016).

3.3.3 Development

The implementation of the full experimental setup was written using Python 2 and run on an Ubuntu Linux system for full compatibility between the framework and dependencies.

The structure was python script that could set up the environment for the OpenAI gym game, build a neural network using TensorFlow, capture the data from the running game, pass that data through the neural network, and have the network output an action to use for that game frame. The game would then be played for that frame, the observations for that frame captured, and passed to the network again to check the results of the previous action and output another action for the next frame.

This would be performed for a set amount of iterations, with the neural network learning via the observations made from the game with observations being logged (score, number of lives, current iteration), and would be used to compare the performance of the trained neural network to others that had been trained with different techniques, with the same number of iterations.

The network structure was then constructed using standard TensorFlow functions with an input layer, 2 convolutional layers, 1 hidden layer, and the final output layer. The script would then build 2 networks, a target and an online, with the target paying the game, and the online network learning from the output of the game and being copied at set intervals to the target network. This helps prevent a feedback loop occurring, as the changes made to the network are cumulative, rather than done for every frame of data.

Image processing was inspired by research done into teaching a machine learning system Pong (Parthasarathy, 2016). Using slice notation, the image array data received from the game environment was able to be cropped, downscaled, and converted to greyscale to reduce the amount of data that needed to be processed by the network.

Control experiments were done after significant changes to the script, to ensure that the integrity of the network was maintained through training, and that learning was still observed in control conditions. This was needed to make sure that changing parameters would be valid experiments.

3.3.4 GPU Acceleration

TensorFlow comes precompiled to work on the widest range of system available, and due to this leaves out many optimisations that can improve training time for specific hardware.

By recompiling TensorFlow from its source using the Bazel build system, optimisations were able to be added at build time such as MKL or AVX, allowing faster processing, with the downside of less compatibility (TensorFlow, 2018b).

During initial setup and testing, training using a CPU only build of TensorFlow proved to be extremely slow, with a full training run potentially taking more than a week to complete. To combat this, TensorFlow GPU was installed instead, along with CUDA, and cuDNN to provide GPU acceleration to the training. This setup proved challenging as different versions of CUDA, cuDNN, and TensorFlow were specific on dependencies and each other, with some later versions not compatible with older graphics cards with a lower version of CUDA compatibility.

However, when full set up, the GPU acceleration provided 3-4x faster training times, cutting total training down to 2-3 days.

3.3.5 Tools

The main development environment was Ubuntu 16.04.4 LTS (Xenial Xerus) (Canonical Ltd., 2018), with Atom as the main text editor for the project (GitHub, 2018). The LTS version of Ubuntu was chosen due to its compatibility between the frameworks used, and also the dependencies which were required by TensorFlow, OpenAI gym, and GPU acceleration.

A number of python modules were used such as matplotlib (Matplotlib, 2018) and numpy (NumPy, 2018). These were required for plotting the game's screens, and for advanced array functions respectively.

Python was the main development language (Python Software Foundation, 2018), specifically Python version 2.7.13. The use of Python 2 over Python 3 was due to

compatibility issues that could arise from combining TensorFlow and CUDA, as well as their related dependencies.

For assisting in development, planning tools like Trello and Bitbucket were used to keep track of issues and tasks. Git was used as version control manager, with Bitbucket serving as a remote repository to assist in backing up data.

3.3.6 Performance and RAM

Another aspect of network performance was the data used as input to the neural network. With OpenAI, it was possible to get the full screen data from the game, along with score and lives. This was passed to the network unmodified from the game initially.

However, this approach caused a large amount of RAM usage, due to needing to store previous states with the actions the network had taken. It was also reducing the performance of the network, due to the large amount of data, a 210x160 pixel image, with a 128-colour palette. Instead of this, by pre-processing the image by converting it to greyscale, downscaling, and cropping it, the image was reduced to a greyscale image with dimensions of 88x80, reducing the total number of data point from 37800 to 7040, an 81% reduction in the amount of input data, with no significant effects on the network's learning rate based on some pre-experimental tests. This same method was very similar to one discussed in the literature review, to avoid giving the network unnecessary data (Mnih et al., 2013). This allowed a network previously capable of using up to 16GB of memory (system limit of RAM and Swap memory) and then being terminated by the system, to using on average around 7.5GB of RAM.

3.4 Summary

This chapter covered the design and implementation of the neural network. Using TensorFlow's machine learning framework, it was possible to program a deep learning neural network in Python. From there, setting up an OpenAI gym emulator to able to send and receive data from the network would allow the game to be played. By taking those observations from the game and recording how the input affected the state of the game, the network could then be trained to achieve a goal. Changing aspects of the network, such as the overall goal that the network should achieve, would be used to determine which techniques were most effective in teaching the neural network how to play the video game.

The next chapter will explore the experiments done to determine the best techniques and strategies used to effectively train the network.

4 Investigation and Comparison of Machine Learning Techniques

4.1 Experiment 1 – Initial Control, Ms. Pac-Man

4.1.1 Description

The initial experiment was performed as a control experiment to provide a baseline for future experiments with Ms. Pac-Man and to see what issues would arise when performing training.

The main aspects of the network were set up according to default and tutorial values provided by TensorFlow (TensorFlow, 2018c), with the assumptions that these would be suitable for training. Other values were chosen on limitations of the hardware ascertained by running a very limited run of the network, with the assumption that these could be changed if it appeared to be affecting the experiment.

Table 4.1 Variables setup for initial experiment.

Variable	Value
Learning Rate	0.001
Replay Memory Size	450000
Batch Size	45
Total Iterations	4000000
Epsilon Decay Iterations	2500000
Epsilon Max	1.0
Epsilon Min	0.1
Discount Rate	0.95
Momentum	0.90
Kernel Sizes	(8,8) (4,4) (3,3)

Table 4.1 shows the raw value of these variables. A brief explanation of the variables is:

- Learning Rate: The rate at which the individual weights in the neural network nodes are adjusted, and therefore how quickly the network changes in training. Larger values mean larger changes.

- Replay Memory Size: The number of previous frames of game data to use for learning. The network will store this many frames of game data and use them to adjust the weights of the network during evaluation. The higher the number, the higher the RAM usage of the network.
- Batch Size: The number of game states taken from the replay memory per iteration and used to evaluate train the network and adjust weights. Larger values mean longer processing time.
- Total Iteration: The number of training iterations that the network will do before completing training. This value is easily adjusted up and down based on how long the training should take.
- Epsilon Decay Iterations: Epsilon in Q Learning is used to represent how "random" the actions the network takes are. Higher Epsilon values mean the network will take more random actions, and lower values mean the network will use its own evaluation to provide actions to the game. The Epsilon Decay Iterations is how many iterations the network will take to get from Epsilon Max to Epsilon Min.
- Epsilon Max: The maximum value of Epsilon. At values of 1, 100% of the actions taken will be random.
- Epsilon Min: The minimum value of Epsilon. At values of 0.1, 10% of the actions taken will be random.
- Discount Rate: The amount by which future reward is discounted, which can make the network prioritise immediate rewards rather than future ones. A discount rate of 0 means the network will only consider immediate rewards.
- Momentum: Momentum assists in the gradient descent of a neural network. As a value changes, momentum ensures that the descent occurs in the same direction unless a large amount gradient occurs in the opposite direction. This prevents the network from rapidly changing weights based on small amounts of data. The momentum value is how much previous weight changes affect the current weight change.
- Kernel Sizes: Due to using a convolutional neural network, data is required to be processed in small groups, or kernels. The size of these kernels affects the "precision" of the network, with larger values beings fuzzier to the network. This should then be passed to increasingly precise layers. Here, the kernels

represent the kernel sized in each subsequent layer, getting smaller as they go, meaning the network first gets a large overall impression of an image, and then finer and finer details.

The main objective was not to optimise the network at this stage but provide a baseline from which to compare other experiments.

Here the basic set up was a static learning rate, a simple reward heuristic (total score), and observations being passed to the network as a reduced scale greyscale image. The game was first trained for 2000000 iterations to see the initial effects of training on the network and then trained further to a total of 4000000 iterations.

4.1.2 Challenges

Some challenges found while implementing and running this initial experiment needed to be solved to allow for the rest of the experiments to be meaningfully compared. These were long training times, lack of comparable data, and potentially incorrect data.

One of the initial challenges with the experiment was that the artefact was running on a CPU. This meant that the training time until the network produced a semi-competent player AI was around a week of continuous training. This was solved by implementing the GPU version of TensorFlow, allowing acceleration provided by a graphics card. This managed to cut the training time of the network down to 2 days.

In addition to processing power constraints, the system the network was trained on had 8GB of RAM. While running, the network often took up to 7.5GB of RAM, leaving little left for the system. Experimenting with larger sizes caused the program to use swap memory in Ubuntu, which slowed down training considerably, and large sizes to use all RAM and Swap, leading to termination by the system. While this was partially solved by the pre-processing of screen images as explained in development, this meant that the total memory of the network was to be constrained to allow for faster training times.

Another challenge was the unknown training time, and how to properly track the AI's progress, to see if it was learning at all. This was partially solved by outputting information such as total score, loss, and q values at regular intervals into a CSV file which could then be graphed. TensorBoard, a visualisation tool for TensorFlow, was considered, but due to the RAM limitations, could not be run continually. The unknown

training time was determined by this initial experiment, and after optimisations listed above, determined to be around 2 days for a fully trained model, which was acceptable for the projects timespan.

4.1.3 Evaluation and Results

The time taken to perform 4000000 iterations was around 30 hours of constant training time on a GPU accelerated system. The time taken to train the model produced an AI agent that was competent at the game, but not any more than a novice player.

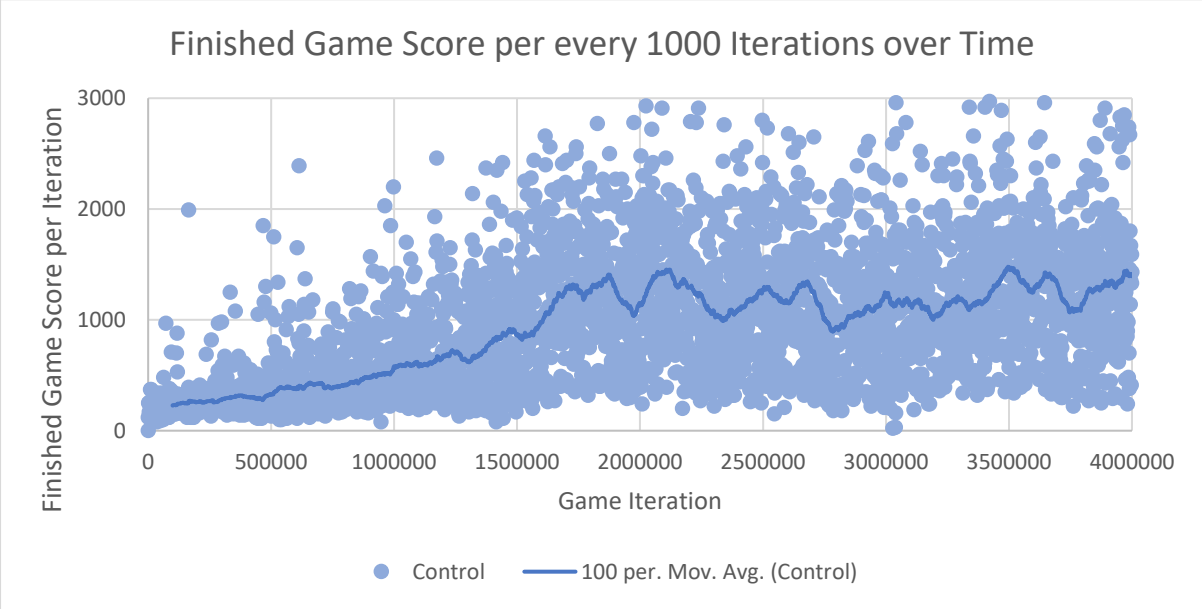


Figure 4.1. Graph of Control Experiment

Table 4.2 Table of Control Experiment Results

Experiment Label	Control
Mean Score (Last 50 Iterations)	1217
Mean Score (Total)	947
Standard Deviation (Last 50 Iterations)	709.84
Standard Dev Total (Total)	640.50
Max Score	5210

A general trend upwards can be seen from Figure 4.1, with a plateau around 2000000 iterations. From Table 4.2, the average score at the end of training using the last 50

values was 1221, and the total average was 947. which was accumulated from traversing around half the playable area. This was a relatively low score, on par with a first-time player of the game.

One notable aspect of the AI's behaviour was the tendency to stick in corners during gameplay, which can be seen in Figure 4.2.

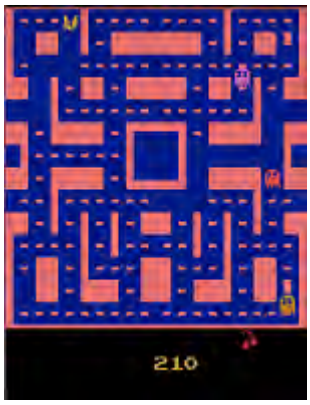


Figure 4.2. An instance of the AI agent playing Ms. Pac-Man reaching a corner and not moving.

This behaviour seemed to be borne out of the rewards given for certain aspects of the game being much higher than others. In this case, those corners contained power pellets, items that gave a larger score when collected. And, as part of the effect, turned the ghost's edible. Eating those ghosts also gave a much larger score

4.1.4 Conclusion

The control experiment provided a baseline for future experiments, highlighted issues with training that were able to be corrected and highlighted "quirks" of the AI that could be used as the basis for future experiments.

Here, the tendency for the AI to stick to corners was likely due to the large amount of reward it received from eating vulnerable ghosts, a reward which far outstripped the reward for simply eating pills. Thus, the next line of experimentation was decided to focus around adjusting those reward values.

4.2 Experiment 2 – Reduced Large Rewards

4.2.1 Description

From the previous experiment, it was observed that while playing Ms. Pac-Man, the network favoured keeping the player in the corners of the map, likely due to the large rewards gathered there.

For the next experiment, the rewards were adjusted so that any large rewards were reduced to 50. As an example, before the change, each pellet on the board was worth 10 points, each power pellet was worth 50, and each eaten ghost was worth 200 to 1600 points depending on how many had previously been eaten.

The hypothesised outcome was likely a reduction in the behaviour of staying in the corners of the map where the power pellets were, and hopefully a more varied traversal of the map.

4.2.2 Challenges

The main challenge of this experiment was the implementation of changing the rewards behaviour. Due to the way data was extracted from the game, simply changing the number of points received from large score objects could not be done. Instead, when the score was received it was checked to see if it exceeded a certain value, and then was adjusted downwards if it was too large and given to the network with as if it was the lower score.

While this achieved the desired effect, an oversight meant that the logged data for the overall score was now inaccurate as it did not accurately reflect the in-game total score. This led to the initial experiment needing to be rerun with a correction to the code allowing for a "total score" which was the actual score and a "total perceived score" which was the score that was given to the network.

4.2.3 Evaluation and Results

At the end of the training, the Reduced Large Reward experiment was able to outperform the Control experiment in several ways.

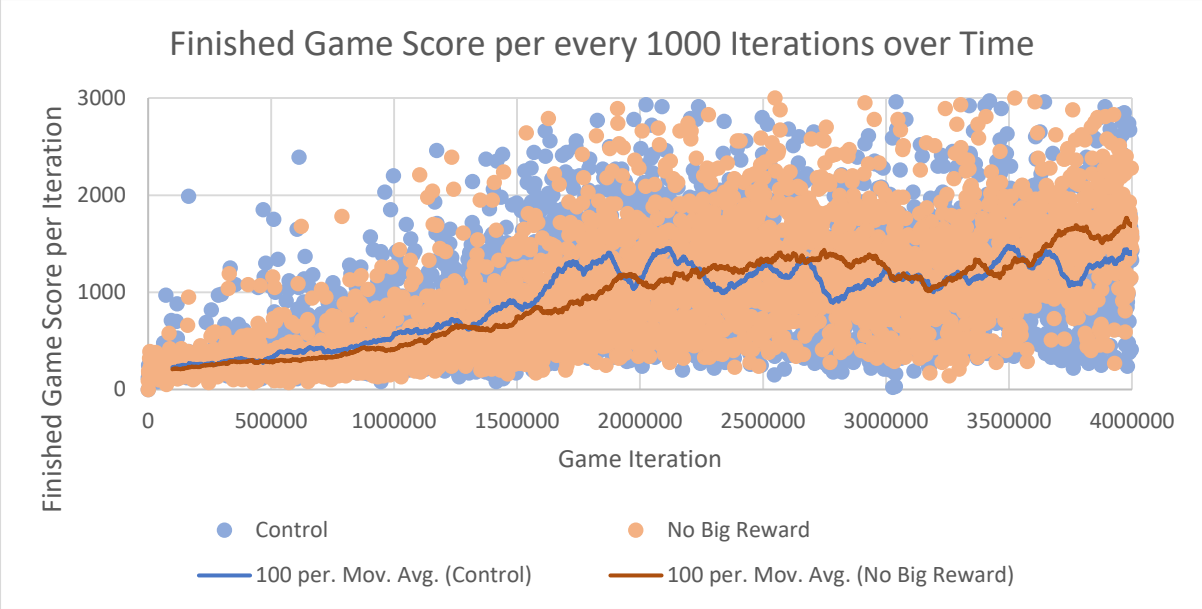


Figure 4.3. Graph of Control vs Reduced Large Reward Experiment

Table 4.3 Table with Results of Control vs Reduced Large Rewards Experiment

Experiment Label	Control	Reduced Large Reward
Mean Score (Last 50 Iterations)	1463	1595
Mean Score (Total)	935	1014
Standard Deviation (Last 50 Iterations)	745.45	606.29
Standard Dev Total (Total)	637.97	656.99
Max Score	5210	6380

Figure 4.3 shows a much smoother ascent than the control value. Table 4.3 shows that the Mean Score for last 50 iterations and Max Score of the Reduced Large Rewards network was significantly higher than the Control experiment, while the actual rate of learning did not significantly increase. The Standard Deviation also shows a more consistent AI than the Control, with less deviation in the last 50 iterations.

The behaviour the AI displayed was also consistent with the hypothesised actions, with Ms. Pac-Man spending less time in the corners of the maze. However, during the later parts of the game, with fewer pills available, the behaviour of sticking to corners was still observed.

4.2.4 Conclusion

This tweak to this reward parameter had small but significant effects on the network. While it had a higher average trained score, it also had much more consistent games.

This likely demonstrates that small tweaks such as changing part of one parameter can have large effects on network training. In the overall training of this reinforcement learning network, this also demonstrates that having large rewards can skew the network into unexpected behaviours.

To experiment with this conclusion further, additional changes to the reward function were tested.

4.3 Experiment – No Large Rewards

4.3.1 Description

This experiment built off the previous two experiments, to see if giving the same reward for all tasks would affect the AI even more, and lead to more advantageous behaviour.

This was done by taking the same setup from Reduced Large Rewards, and then scaling all rewards larger than 10 down to 10, which was the same reward for eating a pellet.

4.3.2 Challenges

One challenge that occurred at this time was the availability of training hardware and subsequently the training time. For the network to be trained in a reasonable amount of time, the GPU, CPU, and RAM of the machine needed to be relatively high-end. Initially, all these experiments were run on a personal high-end gaming PC, but due to other requirements, was also needed for game development and 3D modelling. This meant that training time often had to be stopped in place of other tasks, meaning training often took longer than expected.

This combined with a seeming plateau of results at around 2000000 iterations, meant that it was more efficient to cut training iterations down to 2200000 after this experiment, with little changing in the observed results.

To solve the hardware issues, a request for use of University hardware was made, but due to the requirements of needing to be able to be accessed remotely, and requiring root access, there were no resources available to run the training. An alternative solution was found, by purchasing an old second-hand computer that had a CUDA

compatible graphics card. While the processor, graphics card, and RAM were inferior to the previous training machine, after set-up of remote access, the training framework, and with reduced training times and able to always be running meant that fully training the network took about the same time, a total of 2 days, but was much more convenient.

4.3.3 Evaluation and Results

From testing observations, the AI agent largely did not get stuck in corners, instead preferring to traverse parts of the maze that still had pellets left. This effect also continues to the results of the experiment.

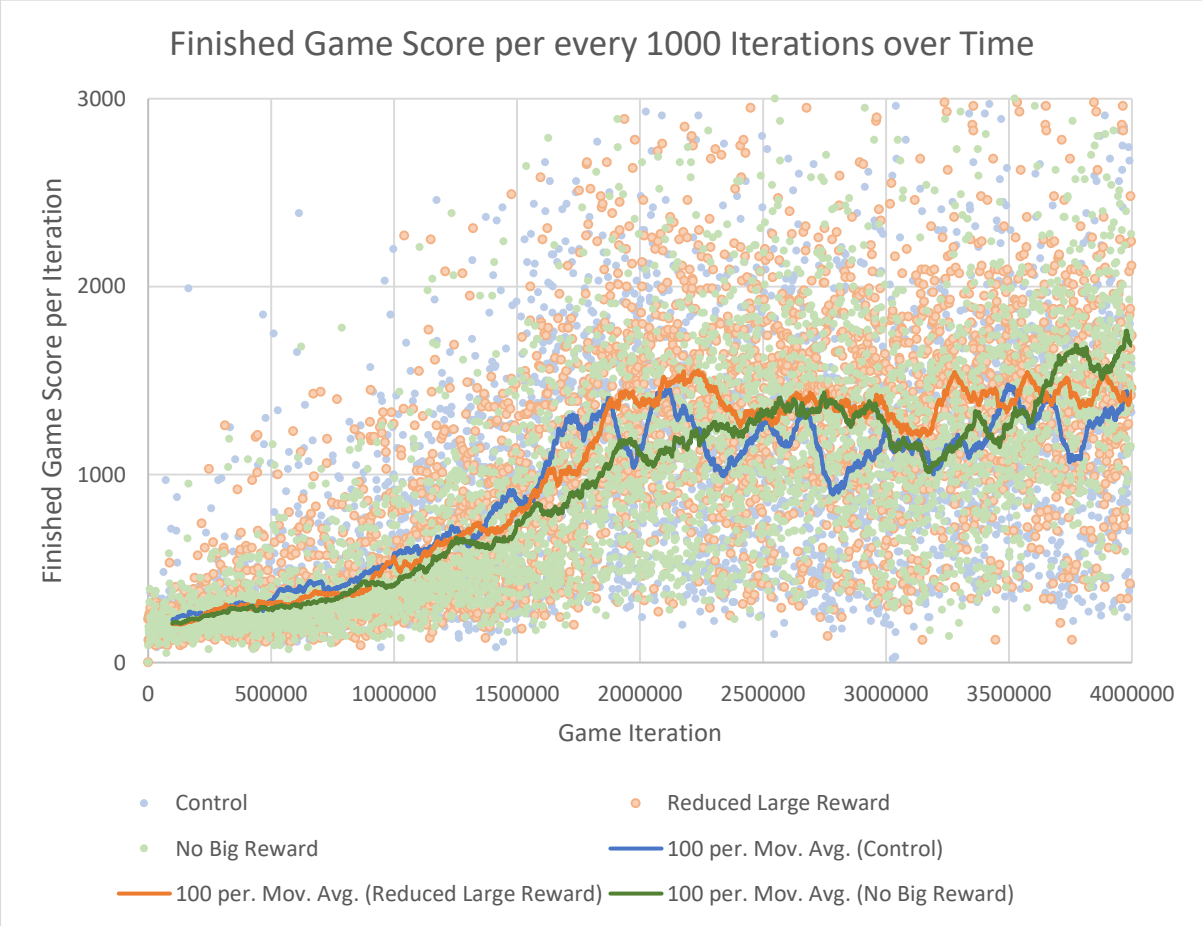


Figure 4.4 Graph of Control vs Reduced Large Reward vs No Big Reward Experiment

Table 4.4 Table with Results of Control vs Reduced Large Reward vs No Big Reward Experiment

Experiment Label	Control	Reduced Large Reward	No Big Reward
Mean Score (Last 50 Iterations)	1463	1595	1647
Mean Score (Total)	935	1014	938
Standard Deviation (Last 50 Iterations)	745.45	606.29	579.80
Standard Dev Total (Total)	637.97	656.99	648.79
Max Score	5210	6380	4800

Shown in Figure 4.4 and Table 4.4, the No Big Reward experiment performed better than both previous experiments in the mean score of the last 50 games. Its standard deviation for the same period was significantly smaller as well, leading to more consistent games played by the AI.

4.3.4 Conclusion

From the previous experiments, in Ms. Pac-Man at least, the large rewards can be distracting to the neural network, weighting those actions as a much higher priority, even though they may not be productive to the overall goal.

However, this is dependent on what the overall goal of the network may be. For example, in terms of learning, the Control experiment was able to train a higher score faster than the No Big Reward experiment, so if the goal was to simply achieve the highest score in the shortest amount of time, the Control setup would be suitable, at least for the first 3000000 iterations. However, the No Big Reward experiment ended with a much higher average score that surpassed the Control experiment, which seemed to plateau at around that time.

So, if defining the overall goal to be the maximum accumulation of points, it appears that tuning the rewards so that the agent favours traversal of the maze appears to be the best way to teach this neural network.

Through all the experiments, the Ms. Pac-Man agent was not observed to avoid ghost, as a human player may do. This led to a large number of deaths which could have

been avoided, but it appeared that the neural network did not seem to have learnt that this behaviour was detrimental overall. The next experiment was a test to see if tweaking the reward system would encourage this behaviour.

4.4 Experiment – Negative Reward for Death

4.4.1 Description

Due to previously explained time constraints, the following experiments were performed to 2200000 iterations instead, as no extra benefit was found from the additional training to 4000000 iterations.

This experiment was an extension from the previous experiments that found that while the network could produce an AI that was able to achieve a decent score, it made no real effort to avoid ghosts, which was the only way for the AI to fail the game.

Thus, to see if this behaviour could be modified, the network was modified to give a negative reward on death, on a setup otherwise identical to the Control experiment.

4.4.2 Challenges

The implementation of a negative reward on death had to be implemented slightly differently than in previous experiments. As there was no score increase or decrease, the only other thing that could be monitored was the screen capture or lives count. As the screen capture was cropped, it would have been difficult to use that to reliably track death on the screen. The other solution was the monitor the number of lives the agent had left, and then apply a negative reward when the number of lives dropped, thus giving a negative reward for the action that led to death. The negative reward given was large at -500, to better show the effects of an exaggerated heuristic.

4.4.3 Evaluation and Results

From observations of the AI behaviour when fully trained, this experiment seemed to have little effect on the overall behaviour of the AI. The AI mostly appeared to follow the same progress as the control experiment, with the same pitfall of over-rewarding corners.

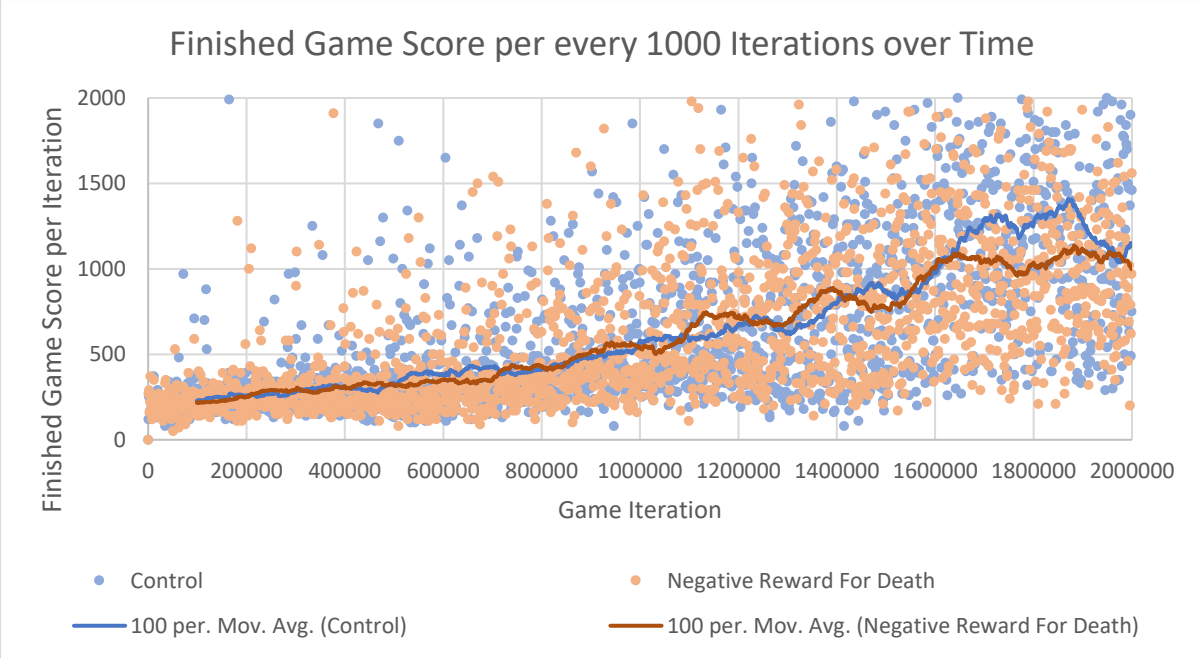


Figure 4.5 Graph of Control vs Negative Reward for Death Experiment

Table 4.5 Table with Results of Negative Reward for Death Experiment

Experiment Label	Control	Negative Reward for Death
Mean Score (Last 50 Iterations)	1243	1031
Mean Score (Total)	722	671
Standard Deviation (Last 50 Iterations)	736.72	433.99
Standard Dev Total (Total)	581.34	507.28
Max Score	5210	5160

From Figure 4.5 and Table 4.5, we can see the average score for both finished and overall were worse than the control values. Otherwise the network performed similarly to the Control experiment, apart from the standard deviation of scores. This mainly means that during training the scores gathered were more consistent, however, this may not reflect the intelligence of the network, and maybe due to the randomness introduced in the network. It may also indicate a plateauing of the performance of the network due to disadvantageous training techniques.

4.4.4 Conclusion

Here, the negative reward had no perceivable effect on the stated goal, to induce ghost-avoiding behaviour in the neural network. This is could be due to a broad range of reasons, for example, the reward over time may have been normalised by the network. Due to it always receiving the same negative reward for a full set of games, it may not have caused any difference in deciding the weight of the actions to be taken.

Another possibility is that the death events were too infrequent compared to the reward events. This means that there were not enough negative reward states for the AI to learn from, meaning that the behaviour was never properly developed in the neural network. This could have been conceivable fixed with more training time, but with no guarantee that the behaviour would emerge.

A third possibility was that the negative reward was not applied correctly to the actions that led to the agent's death, or that those actions were too close to the agent dying so that no action could prevent the AI's death.

For the next experiment, a different way of trying to train different behaviour in the AI was attempted to see if it was more effective, trying to emphasise time spent alive to the AI.

4.5 Experiment – Reward for Time Alive

4.5.1 Description

From the previous experiment, it was determined that the negative reward for discouraging the AI from dying did not cause any observable change in behaviour. A different approach was attempted where the network was instead given increasing rewards for staying alive.

The hypothetical outcome of this experiment was for the AI to not any better than the control, but to emphasise actions that kept it alive.

4.5.2 Challenges

Like the negative reward for death, applying a different reward required a reworking of the reward part of the network. Here, it was fairly simple, after every game frame, a reward was given to the AI, and the reward was increased by 1 point every 4 frames. When the AI died, this cumulative reward would be set to 0 to count up again.

Otherwise, the rewards and parameters for the experiment were the same as the control.

4.5.3 Evaluation and Results

In contrast to the other experiments, which performed near the same or better than the Control, this experiment performed significantly worse.

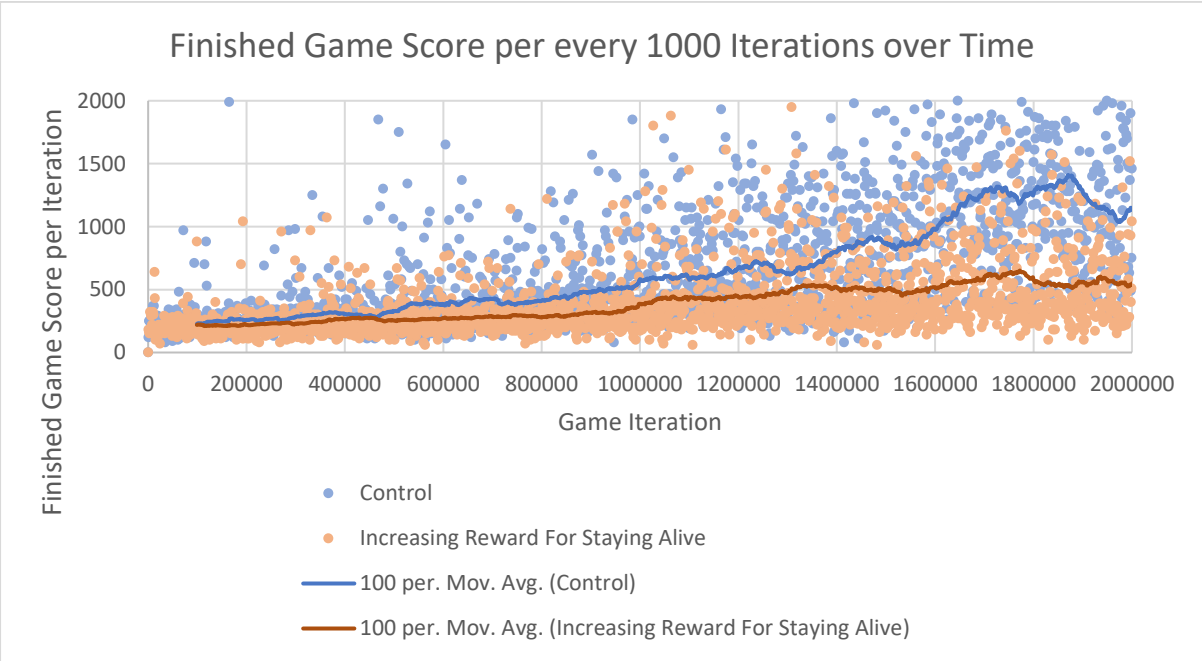


Figure 4.6 Graph of Control vs Reward for Time Alive Experiment

Table 4.6 Table with Results of Reward for Time Alive Experiment

Experiment Label	Control	Increasing Reward for Staying Alive
Mean Score (Last 50 Iterations)	1243	598
Mean Score (Total)	722	408
Standard Deviation (Last 50 Iterations)	736.72	344.39
Standard Dev Total (Total)	581.34	306.00
Max Score	5210	3780

From Figure 4.6 and Table 4.6, we can see the mean and total scores were almost half that of the Control experiment, with standard deviation being the same overall

percentage of the score. In addition, the actual behaviour of the AI agent was not as intelligent as the previous experiments, with Ms. Pac-Man often running into walls and then not moving, in addition to no avoidance of ghosts.

4.5.4 Conclusion

The likely explanation for Ms. Pac-Man moving into walls and then not moving is likely how the network stores states and actions. Due to every action seemingly getting a reward, no matter what the state, the network instead chose actions semi-randomly, and due to starting actions being similar, likely reinforced a behaviour of moving into the direction of a wall. Due to the continual rewards for this, that action was reinforced in the network, meaning that the AI behaved contrary to the overall goal of the game.

Overall, these experiments were unsuccessful in improving the general behaviour of the AI. Instead, more emphasis was put on changing the input data, structure, and learning of the network, rather than changing the reward behaviour.

4.6 Experiment - Colour

4.6.1 Description

In previous experimental setup, it was determined that the trade-off required in memory used was too much to allow the network to process a full-colour image. In addition, the colours would mean that each pixel would include 3 data points instead of just 1 for a greyscale image.

Nevertheless, an experiment was prepared to see if having colour was an important aspect the network needed for better learning.

4.6.2 Challenges

This change was a larger fundamental change to the network. As the network was expecting an 88x10 greyscale image, which translated to an array of single greyscale values, it had to be adapted to fit a colour image, which was given as an array of 3 element array representing red, green, and blue values. In addition, simply taking the image from before converting it to greyscale proved to be untenable, as the memory usage ballooned due to the increase of data.

Due to the way the network was configured, a large memory of previous states was required, along with the actions taken by the network, to analyse states and determine if the expected result of an action. Here, the colour network took up enough memory

per state that it was using all 8GB of RAM and 8GB of Swap memory only 10000 iterations in, causing the training computer to freeze or terminate the program.

To alleviate this, the image was given a different type of pre-processing than the greyscale image. Figure 4.7 gives an example of this greyscale image processing.

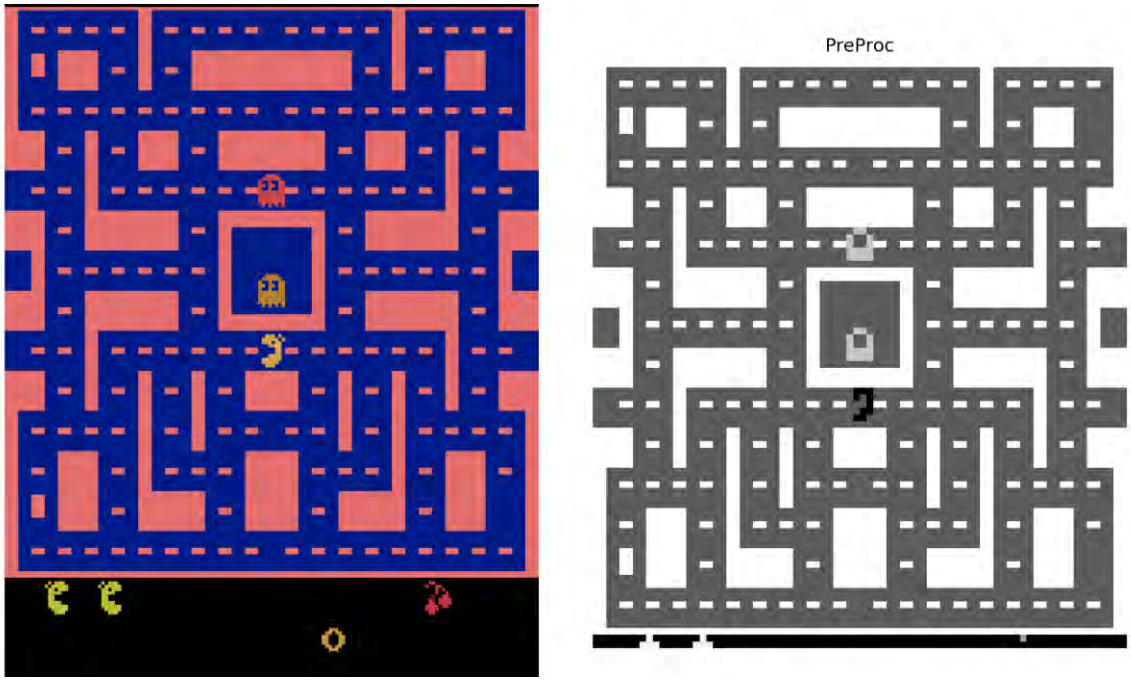


Figure 4.7.The original game screen, along with the greyscale pre-processed image.

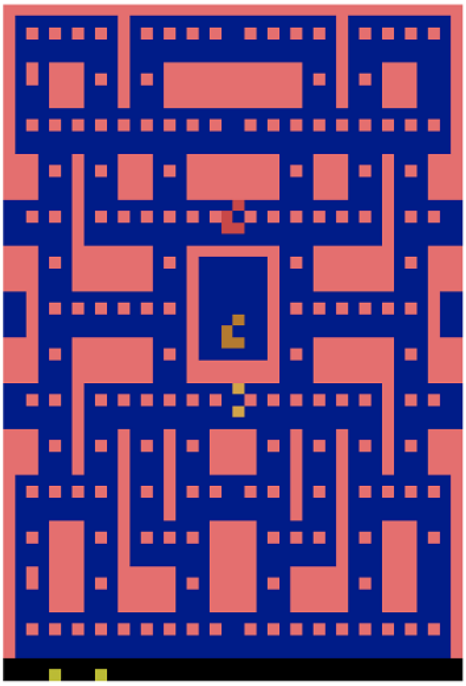


Figure 4.8. An example screen from the colour pre-processor, shrinking the image down further.

Figure 4.8 shows the new image that was scaled down even further to 59x40 pixels, the bare minimum found to still show all the major details of the game, without obscuring some potential details. Here, each pellet was as big as one pixel, except for power pellets which were two pixels.

This shrinking gave a total of 7080 elements for the network to process (59 * 40 * 3) compared with 7040 from the greyscale image (88 * 80 * 1). While similar, the slightly increased amount of data meant that the size of the replay memory had to be shrunk to allow the full program to fit within RAM.

4.6.3 Evaluation and Results

The network through testing appeared to perform similarly to the control value, with the same behaviour observed. However, the network learned more slowly than the control.

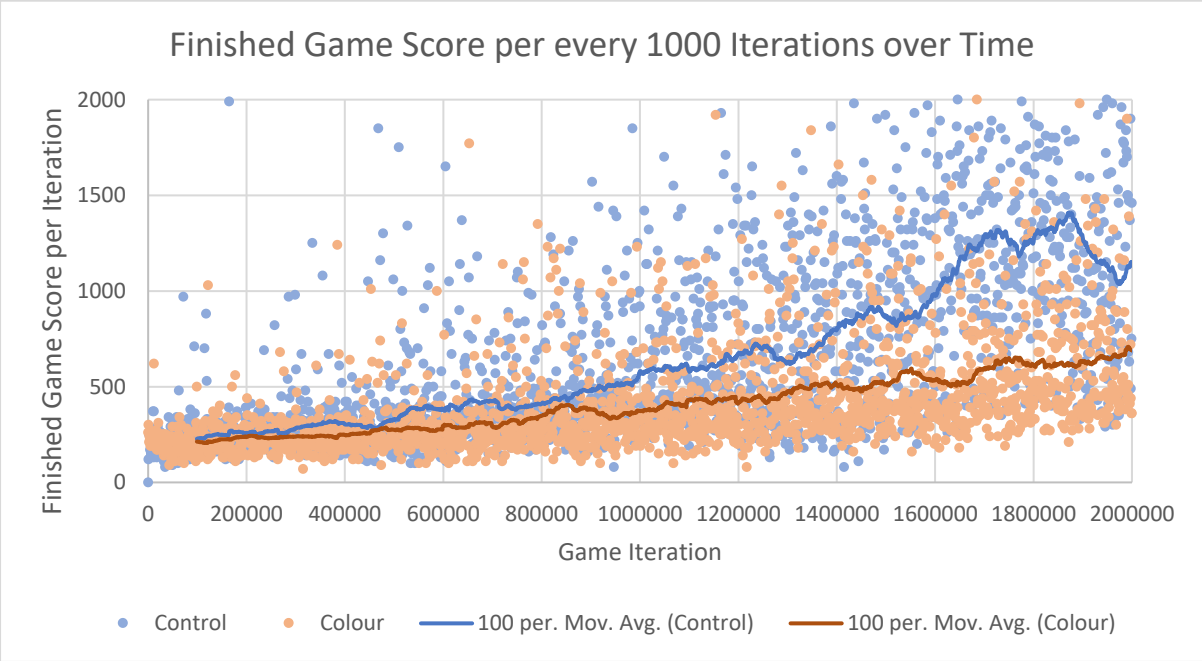


Figure 4.9 Graph of Control vs Colour Experiment

Table 4.7 Table with Results of Colour Experiment

Experiment Label	Control	Colour
Mean Score (Last 50 Iterations)	1243	848
Mean Score (Total)	722	446
Standard Deviation (Last 50 Iterations)	736.72	501.62
Standard Dev Total (Total)	581.34	334.89
Max Score	5210	3750

From Figure 4.6 and Table 4.7, we can see that the scores of the network were consistent with the Control experiment, but only across twice the amount of learning time. This implies a trend similar to the control experiment, but a much slower learning rate.

Observations from testing the network showed behaviour consistent with the Control experiment at where the average scores were similar.

4.6.4 Conclusion

With no distinct behaviour changes between the Control and Colour experiments, and with no other changed factors beyond the replay memory, a constraint caused by limited RAM, it can be determined that colour is not an essential part of the input the network require to learn how to play the game. As long as there are differences in the greyscale values, it appears that the network can make distinctions between various game elements, or at least, detect changes in the network that it can predict rewards for.

Therefore, the greyscale image processing can most likely be used for future experimentations. To further experiment different techniques, changes to the network were examined instead.

4.7 Experiment – High Epsilon

4.7.1 Description

For the Q Learning network, the epsilon value represented the amount of randomness used for game actions. At high epsilon values, almost all the games actions would be

random, and at low epsilon values, the game would mostly use output actions from the established network.

The epsilon value in the game gradually decreased over 2000000 steps to provide an initially high number of random steps for the game to take, and then decreased it gradually over the whole training cycle. This experiment increased the steps at which the epsilon values decreased to 4000000, meaning that at the end of training actions taken by the network could be random 50% of the time.

The hypothesis for this would either be a breaking of the plateau of the control values, or an overall decrease in effectiveness as the random actions were not beneficial as the network was sufficiently trained.

4.7.2 Challenges

The implementation of this experiment was relatively simple, as the decrease of the epsilon value was already defined in the script, and only required a value change. No other major challenges occurred with this experiment.

4.7.3 Evaluation and Results

From observation of the trained network, the High Epsilon network was not able to train to the same level as the Control experiment in the same amount of time, and overall the network performed worse than the Control.

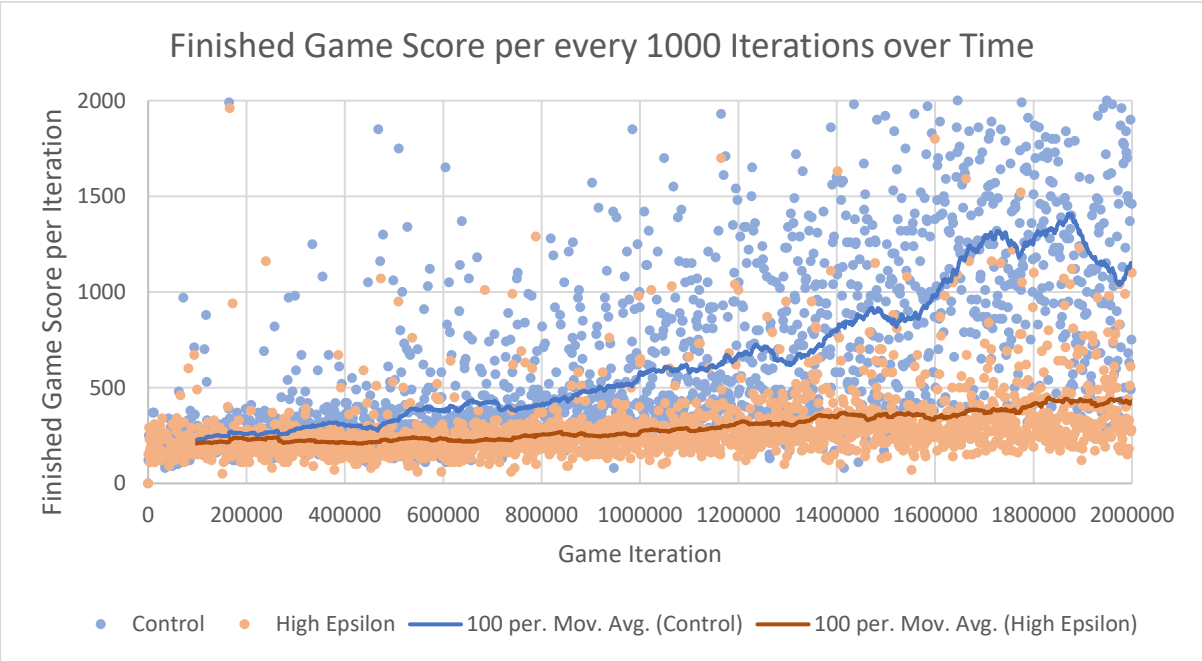


Figure 4.10 Graph of Control vs High Epsilon Experiment

Table 4.8 Table with Results of High Epsilon Experiment

Experiment Label	Control	High Epsilon
Mean Score (Last 50 Iterations)	1243	669
Mean Score (Total)	722	469
Standard Deviation (Last 50 Iterations)	736.72	626.15
Standard Dev Total (Total)	581.34	353.96
Max Score	5210	3990

From Figure 4.10 and Table 4.8 High Epsilon proved to be a mostly ineffective technique to improve learning.

4.7.4 Conclusion

The higher randomness introduced using a higher epsilon value did not improve the network's training in the way hoped. This implies a definite limit as to how much randomness should be introduced into a reinforcement learning network such as this.

A number of different methods were used to experiment with Ms. Pac-Man but did not produce significant results. So instead, the focus was shifted to another part of the project, the experimentation with different games.

4.8 Experiment – Breakout Control

4.8.1 Description

Like Ms. Pac-Man, this version of Breakout was run through the use of the OpenAI gym Atari environments. This experiment was used to set an initial benchmark for future Breakout experiments, and to see if the network was able to train on a different game with a different control scheme.

Like Ms. Pac-Man, the overall goal was the total score, and the rewards were the score from the game. All other parameters were the same as the Ms. Pac-Man Control experiment.

4.8.2 Challenges

Implementing a different game came with some challenges, especially for the pre-processing applied to the game screen. Using the same cropping values as the Ms. Pac-Man led to part of the game area not being visible.

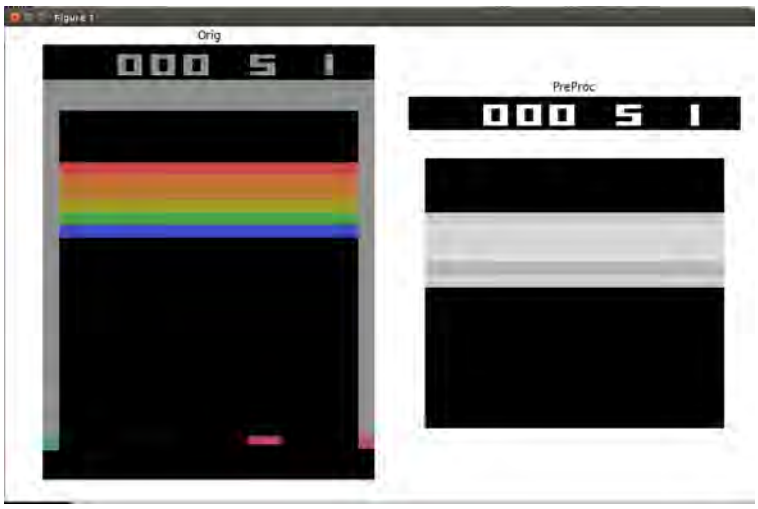


Figure 4.11. A Breakout game screen configured such that the bottom part of the game was not visible.

From Figure 4.11 above, the row with the paddle is no longer visible, which would be essential information for the network. This had to be adjusted slightly so that the cropping showed the entirety of the bottom area of the game and removed superfluous information like the on-screen score, which can be seen in Figure 4.12.

Figure 4.12. Image of correctly pre-processed Breakout screen.

This input allowed a more appropriate and constrained input for the network to train with.

4.8.3 Evaluation and Results

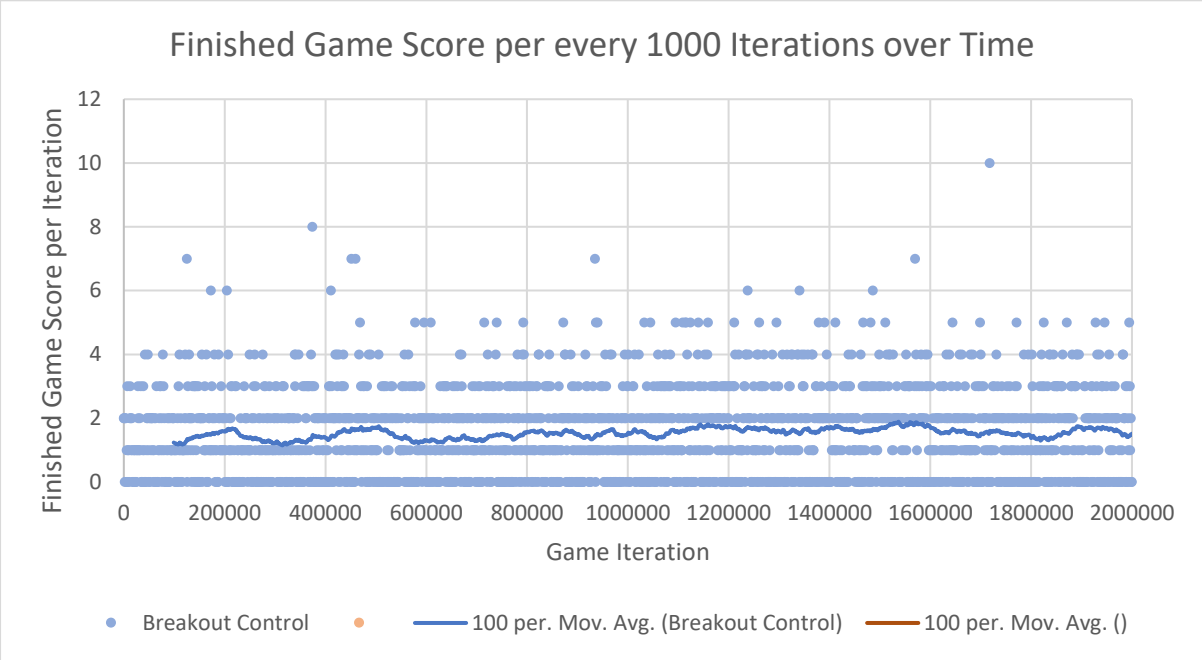


Figure 4.13 Graph of Breakout Experiment

Table 4.9 Table with Results of Breakout Experiment

Experiment Label	Breakout Control
Mean Score (Last 50 Iterations)	2
Mean Score (Total)	2
Standard Deviation (Last 50 Iterations)	1.52
Standard Dev Total (Total)	1.36
Max Score	10

Figure 4.13 shows a mostly flat learning rate, which is also shown in Table 4.9. Therefore, this experiment was unsuccessful at achieving any scores higher than random playthrough. The highest max score can also be attributed to random change.

When testing after completion of training, the network failed to track or even launch the ball required for the game to continue, due to needed to press the fire button to release the ball.

4.8.4 Conclusion

Overall, this experiment was not successful in the training playing Breakout. However, the reasons for this were worth investigating.

An initial hypothesis was that the game's controls were far too different from Ms. Pac-Man to be able to be trained by the network. However, the network was a general Q Learning network and was not specifically designed to train Ms. Pac-Man games.

A more likely reason was that the reward and the actions of the game were very much disconnected. In order to score points in Breakout, a player needs to anticipate where the ball is going to go, move the paddle there, hit the ball with the paddle, and wait until the ball bounces back from the bricks. The last action is the only current action that gives any reward in the current setup, and is not the direct result of an action, unlike moving in a direction in Ms. Pac-Man.

Therefore, to properly train a game like Breakout, an experiment was considered that gave the network more information to be able to make those longer-term decisions.

4.9 Experiment – Breakout Layering

4.9.1 Description

Based on the previous experiment, the current setup of the neural network did not appear to be able to successfully learn to play Breakout. To try and evaluate why this was the case, a different technique was used. A technique where previous frames of an environment were stacked or layered on top of each other, giving each input of the game a small "history" was shown to improve neural network training in certain applications (Mnih et al., 2015). This was implemented in the neural network setup by taking a number of previous frames of the game and then layering that information on each frame of the game passed to the network.

4.9.2 Challenges

The implementation of the layering required more work on the pre-processing of game screens from Breakout. To accomplish the layering effect, the program needed to store previous frames form the game using a queue, keeping a configurable number of frames. The previous frames were then added to the current game's frame after being faded out to indicate that they were past frames. This ended up giving the network an

image of the game screen with an "after-image" of the elements moving in Breakout which can be seen in Figure 4.14.

Figure 4.14. An example of the layering in Breakout.

4.9.3 Evaluation and Results

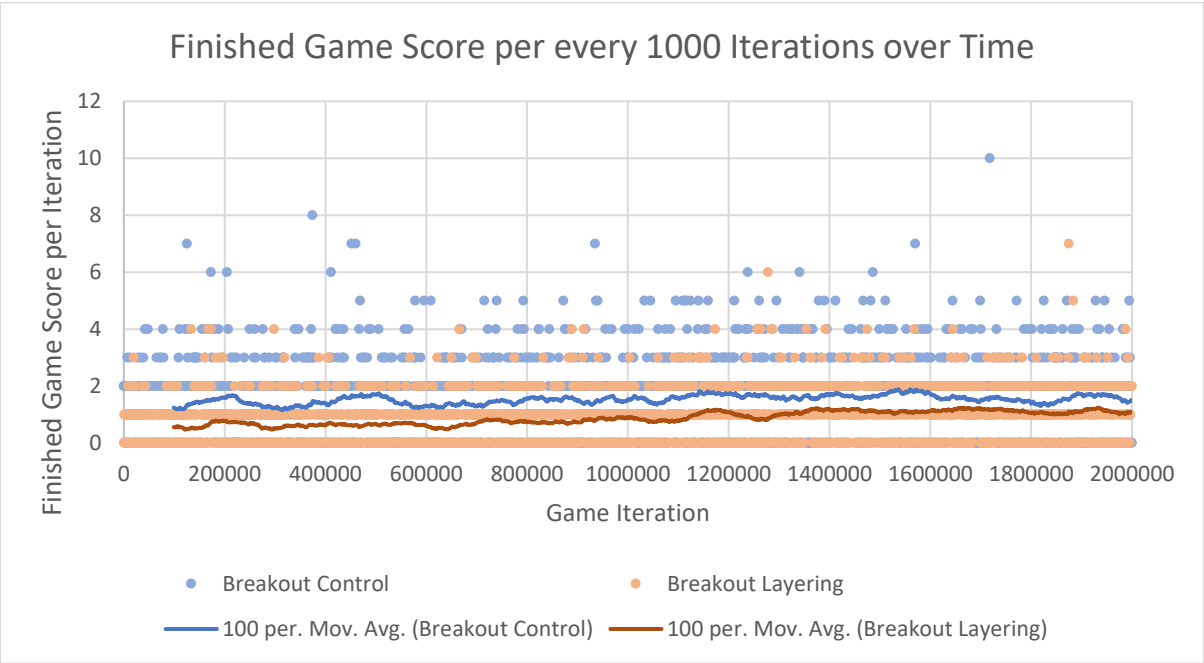


Figure 4.15 Graph of Control vs Breakout Layering Experiment

Table 4.10 Table with Results of Control vs Breakout Layering Experiment

Experiment Label	Breakout Control	Breakout Layering
Mean Score (Last 50 Iterations)	2	2
Mean Score (Total)	2	1
Standard Deviation (Last 50 Iterations)	1.52	1.19
Standard Dev Total (Total)	1.36	1.29
Max Score	10	11

Unfortunately, the layering did not have a significant effect on the training of the network, as shown in Table 4.10, and proved to be worse at playing the game than the Breakout Control experiment. While the results in Figure 4.15 showed an upward trend, this was likely due to the high outliers from random games, rather than a trend of learning.

4.9.4 Conclusion

The possibility that Breakout was simple too different a game to be properly trained by this network was more likely after conducting this experiment.

The hypothesis that the action was too disconnected from the results is most likely the case, as the network would be unable to properly train any actions as it did could not properly correlate a reward with a previous action.

Going forward with other experiments, this was expected to be the case for other games that similarly did not have rewards that tied directly to a reward.

4.10 Experiment – Ms. Pac-Man Layering

4.10.1 Description

While Breakout was not a successful experiment, the layering effect was a proven technique from a research paper, and it was possible that the same technique could produce beneficial effects in Ms. Pac-Man.

4.10.2 Challenges

With the implementation of the Breakout games, it was relatively easy to adapt the same layering effect in the script to Ms. Pac-Man as well.

However, due to the different colours and greyscale values of the Pac-Man games, as well as the Atari emulators flickering of sprites, the layering effect was not fully visible on the Ms. Pac-Man frames.

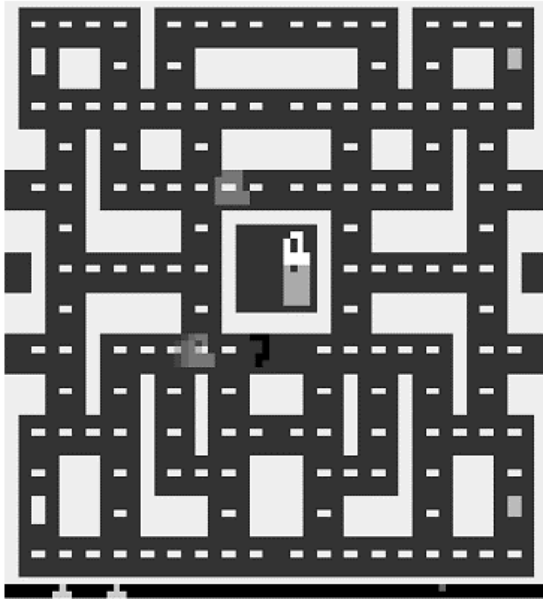


Figure 4.16. An example screen from the layering applied to Ms. Pac-Man.

As shown in Figure 4.16, most of the ghosts do not after the same "after-image" as in Breakout, and their fading is actually applied backwards. However, due to the nature of the neural network, it would still be able to detect the faded colour as the actual position of the ghost. Thus, it was assumed it would at least not cause a degradation in the network's performance.

4.10.3 Evaluation and Results

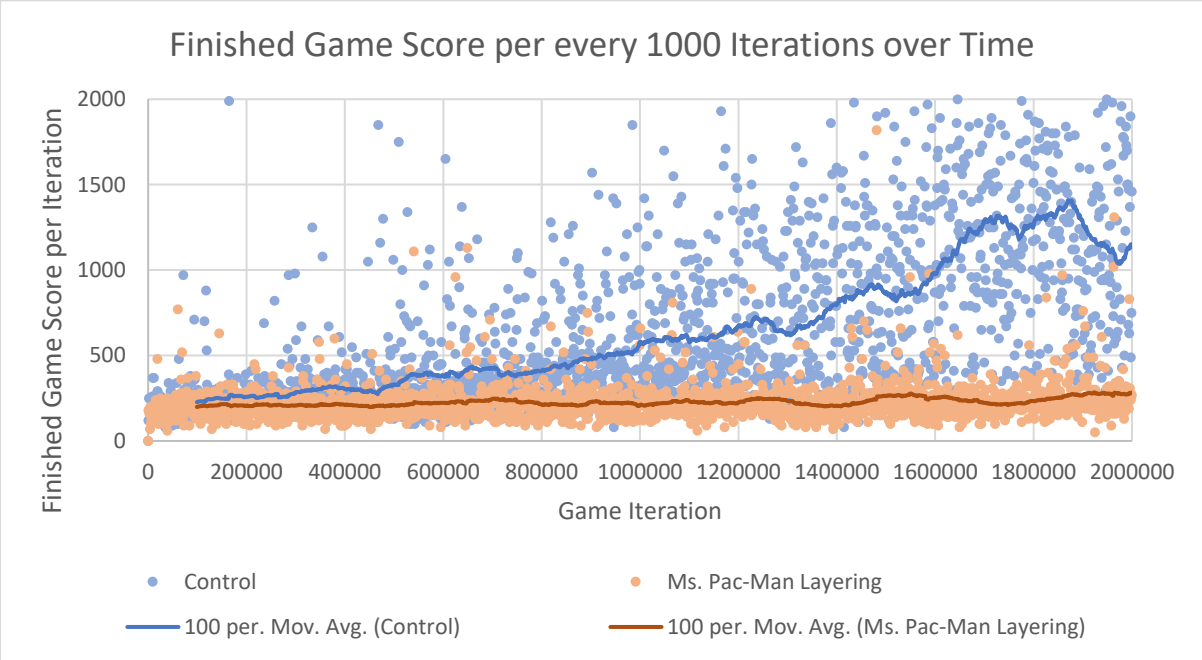


Figure 4.17 Graph of Control vs Ms. Pac-Man Layering Experiment

Table 4.11 Table with Results of Control vs Ms. Pac-Man Layering Experiment

Experiment Label	Control	Ms. Pac-Man Layering
Mean Score (Last 50 Iterations)	1243	280
Mean Score (Total)	722	233
Standard Deviation (Last 50 Iterations)	736.72	131.06
Standard Dev Total (Total)	581.34	117.07
Max Score	5210	1820

Interestingly, the network produced no better results than a random agent, as seen in Table 4.11. No sign of learning or an upward trend was observed as in Figure 4.17, and it appeared as if the layering effect had detrimentally affected the network.

4.10.4 Conclusion

Initially, the reaction to these results was confusing, as even with the layering effects, all information was still available to the network to process. This meant that even with the layering, the performance of the network's training should have been similar.

Near the end of the project, it appeared that while the layering effect worked as expected in visualisation, a bug in the coding of passing the merged image to the network had instead caused it to pass the previous frame of the game. This meant that it was continually passing forward the same initial frame of the game. As the layering was abandoned after these experiments, it was not noticed until after the experiments were no longer being performed.

However, it was unclear whether the initial version of the Breakout Control experiment suffered from the same issue, and there was not enough time to retry the experiments with an implemented fix. Thus, the results of the layers experiments would have to put down as inconclusive.

4.11 Experiment – Other Games

4.11.1 Description

A number of other games were considered after Breakout and Ms. Pac-Man. These games were a mixture of Atari games and were chosen primarily based on their games ability to give rewards (score) for actions performed in the game. Specifically, ones where score and action were either directly tied, or not at all, to see if the similar results could be obtained.

These included Pong, Asteroids, Qbert, Pinball, and Space Invaders.

4.11.2 Challenges

For each game, the pre-processing code needed to be configured to properly crop and resize only the specific portions of the game windows that were required for learning. In addition, these games needed customised start times to cut out superfluous frames of the game where the agent was respawning from a death.

Collecting the data for some games was also difficult, as the scale of the game's rewards differed wildly between games (negative scores for Pong, scores of 100s from Qbert, and scores of 10000s from Pinball).

4.11.3 Evaluation and Results

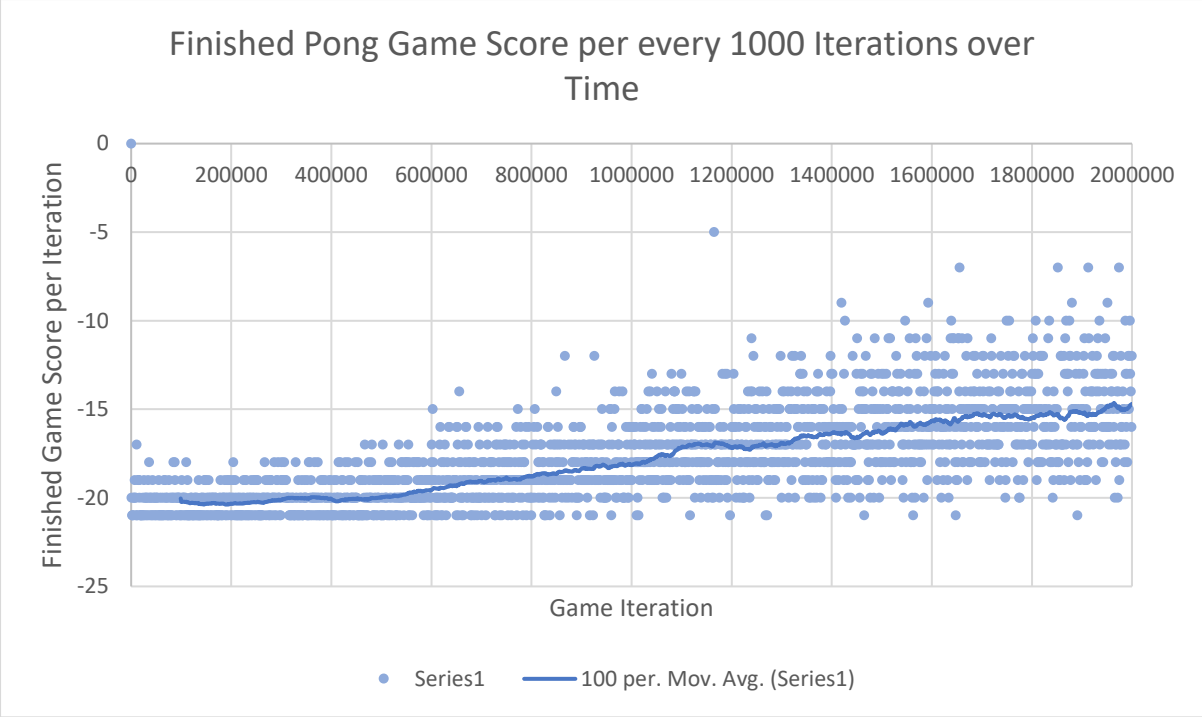


Figure 4.18 Graph of Pong Experiment

Table 4.12 Table with Results of Pong Experiment

Experiment Label	Pong Control
Mean Score (Last 50 Iterations)	-13
Mean Score (Total)	-16
Standard Deviation (Last 50 Iterations)	2.88
Standard Dev Total (Total)	3.44
Max Score	-2

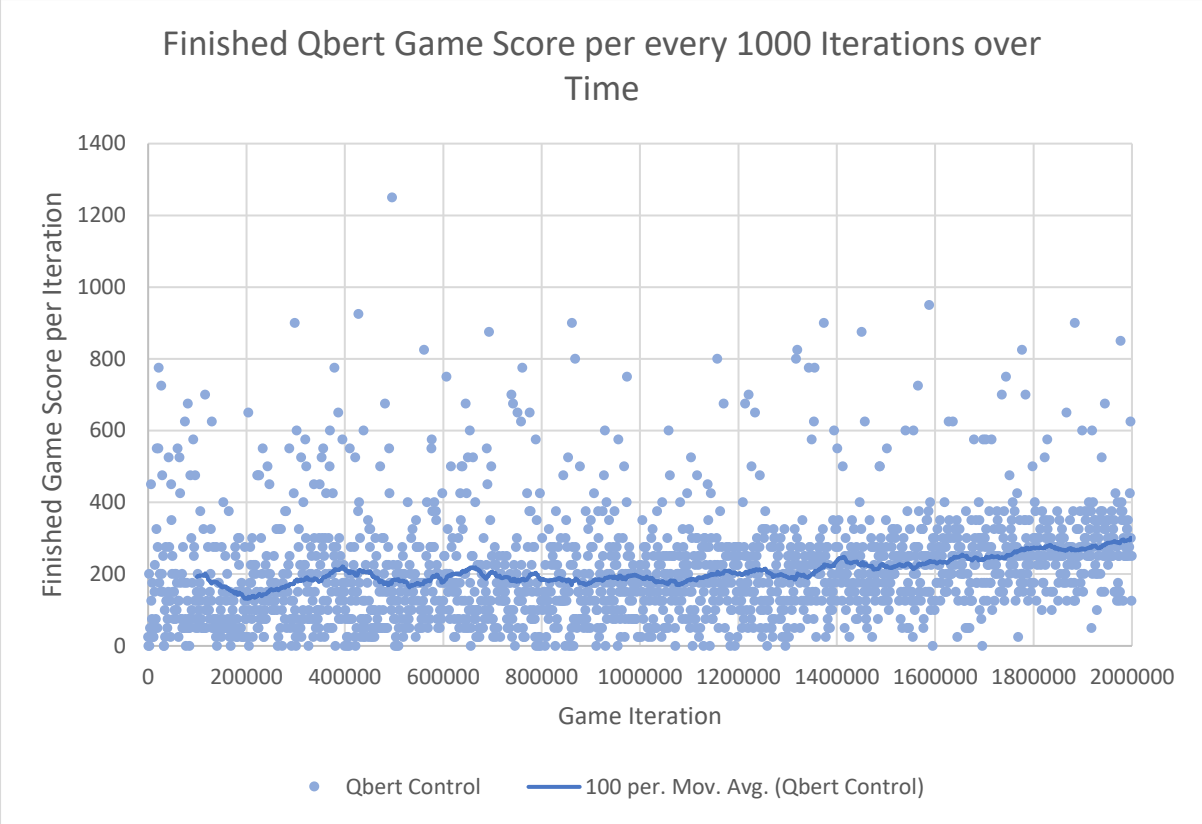


Figure 4.19 Graph of Qbert Experiment

Table 4.13 Table with Results of Qbert Experiment

Experiment Label	Qbert Control
Mean Score (Last 50 Iterations)	269
Mean Score (Total)	257
Standard Deviation (Last 50 Iterations)	74.30
Standard Dev Total (Total)	116.87
Max Score	1250

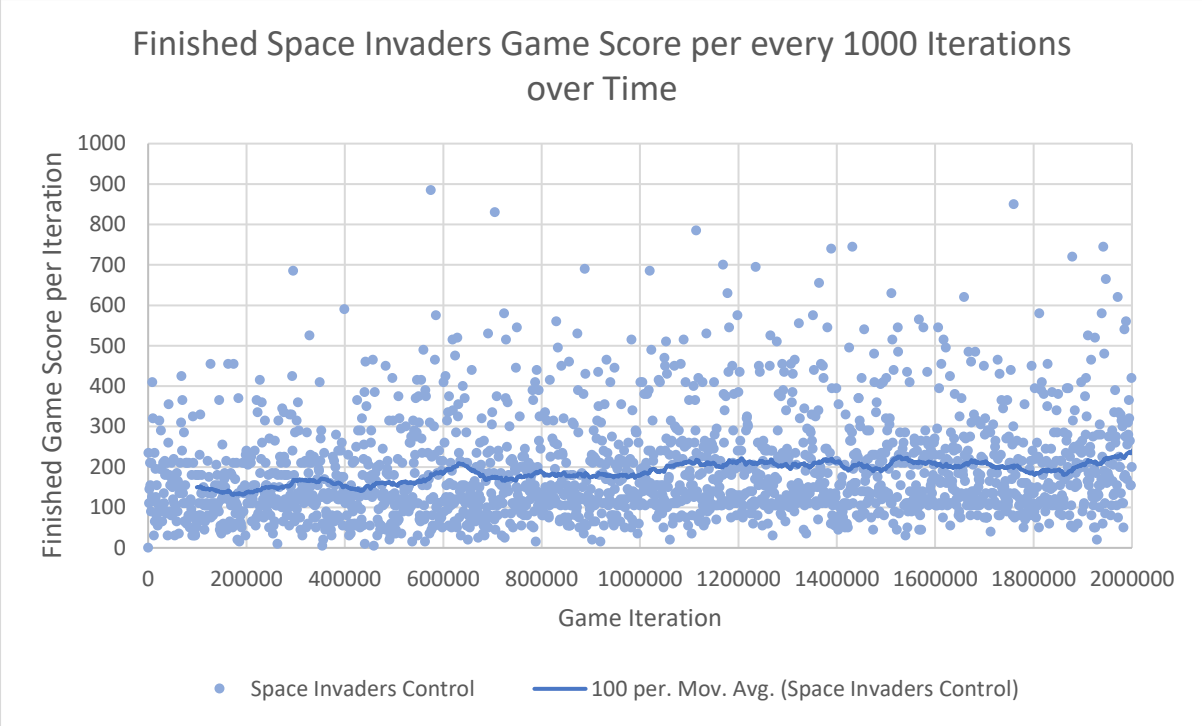


Figure 4.20 Graph of Space Invaders Experiment

Table 4.14 Table with Results of Space Invaders Experiment

Experiment Label	Space Invaders Control
Mean Score (Last 50 Iterations)	208
Mean Score (Total)	190
Standard Deviation (Last 50 Iterations)	109.27
Standard Dev Total (Total)	122.16
Max Score	885

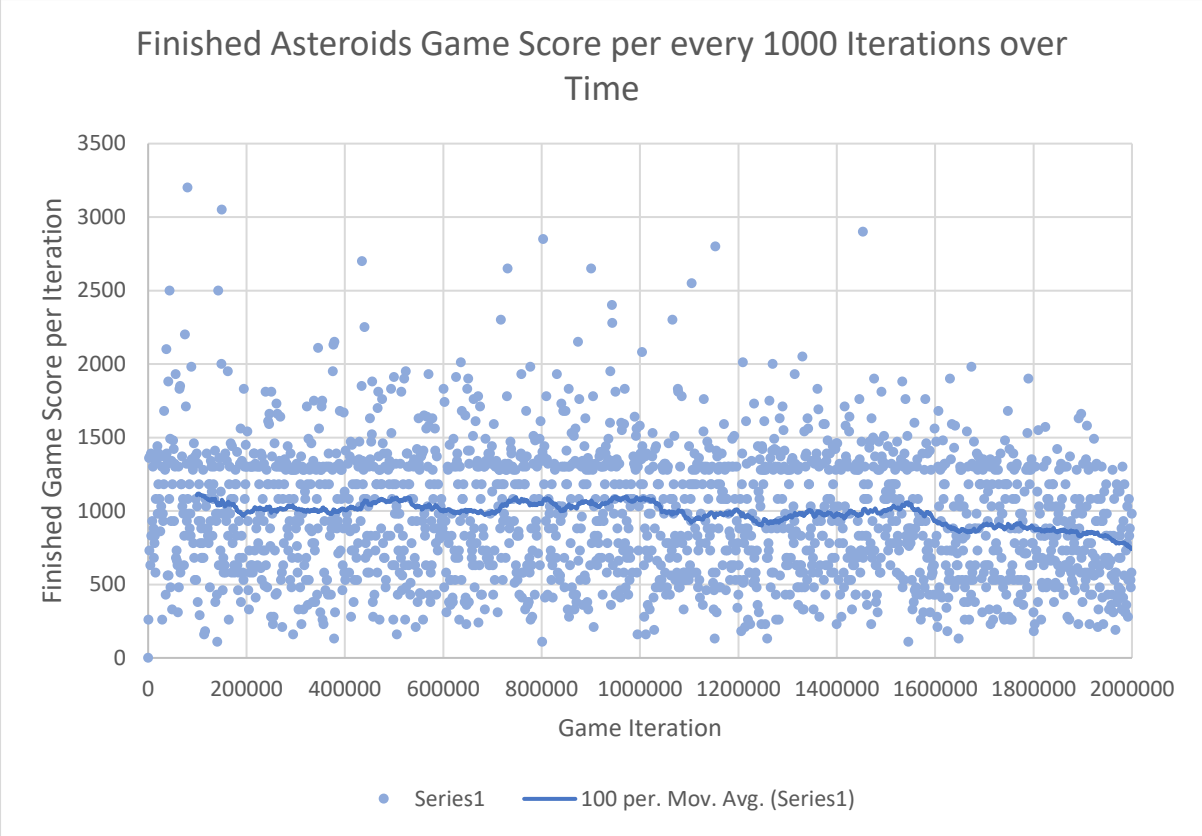


Figure 4.21 Graph of Asteroids Experiment

Table 4.15 Table with Results of Asteroids Experiment

Experiment Label	Asteroids Control
Mean Score (Last 50 Iterations)	769
Mean Score (Total)	865
Standard Deviation (Last 50 Iterations)	329.16
Standard Dev Total (Total)	404.25
Max Score	3200

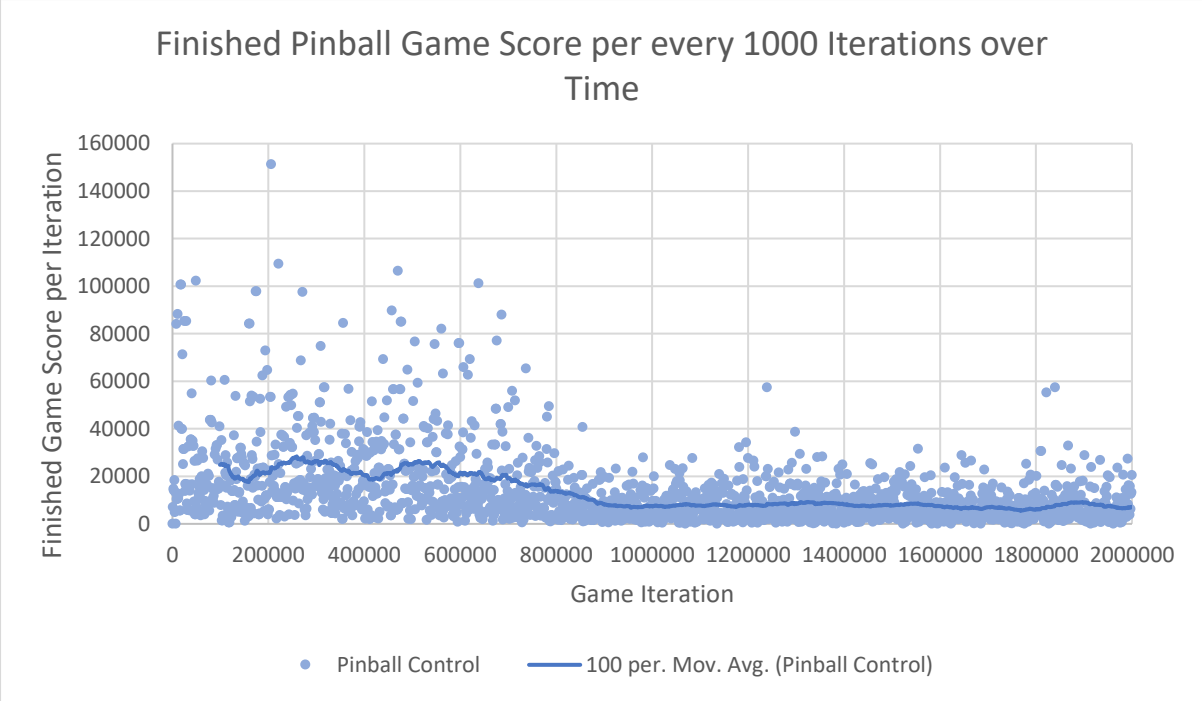


Figure 4.22 Graph of Pinball Experiment

Table 4.16 Table with Results of Pinball Experiment

Experiment Label	Pinball Control
Mean Score (Last 50 Iterations)	6622
Mean Score (Total)	12713
Standard Deviation (Last 50 Iterations)	4606.16
Standard Dev Total (Total)	14653.50
Max Score	151278

Going through the results of these games, an overall trend can be seen, with one notable exception.

In order of games where the training on the network improved the scores of the AI, the games were Pong, Qbert, Space Invaders, Asteroids, and finally Pinball.

Taking these in order, it can be seen there is a correlation between the time the action is taken, and the when the reward is given.

Starting with Qbert, the actions taken in the game are decided where to jump. The movement key corresponds to a block to jump to, which gives an immediate score reward. This correlates with the increase of training ability of the network.

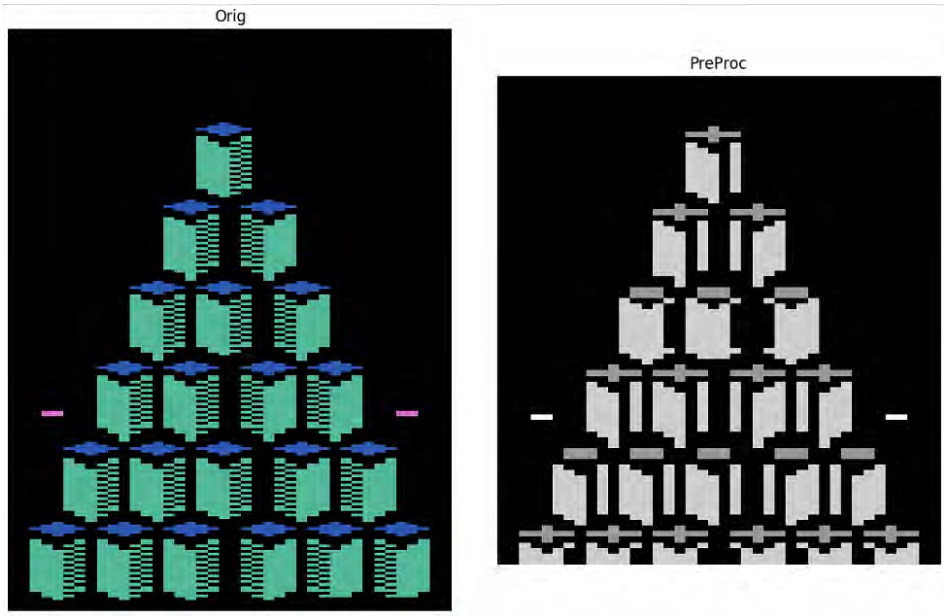


Figure 4.23. The initial frame of Qbert, unprocessed and pre-processed.

The next is Space Invaders, where pressing the fire button will fire a bullet towards the enemies. Due to the bullet's travel time, and the ability to miss, this means that the actions the AI takes do not fully correspond to an appropriate reward, meaning the training correlation for the neural network was fairly weak.

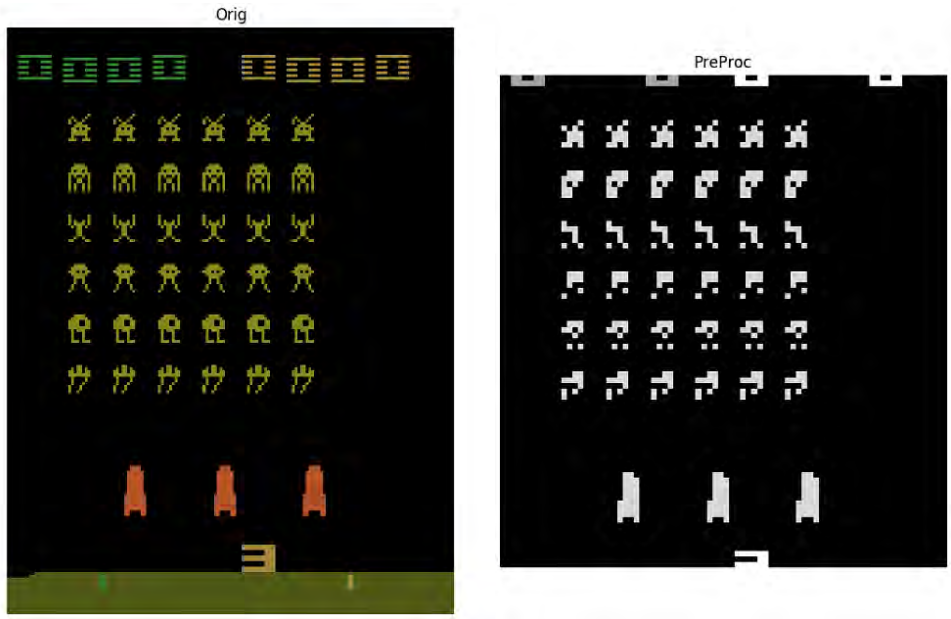


Figure 4.24. The initial frame of Space Invaders, unprocessed and pre-processed.

The next two games move into where the training of the network was actively detrimental to the overall goal of the game.

Starting with Asteroids, a general downward trend of the score is likely due to the controls and nature of the game. Firstly, a bullet being fired may not hit an asteroid for some time, meaning a delayed reward. Secondly, staying alive in the game is a much more difficult task, due to the momentum of the ship and the constantly moving hazards. In addition, the flickering effect where objects disappear in the Atari emulator is much more pronounced in this game, most likely due to the multiple moving elements.

All of this contributes to actions the network takes actually reducing the total score, due to crashing into asteroids. As the network is trained an action become less random, it is not able to unlearn pressing a directional key, which introduces a much higher risk of death in the game.

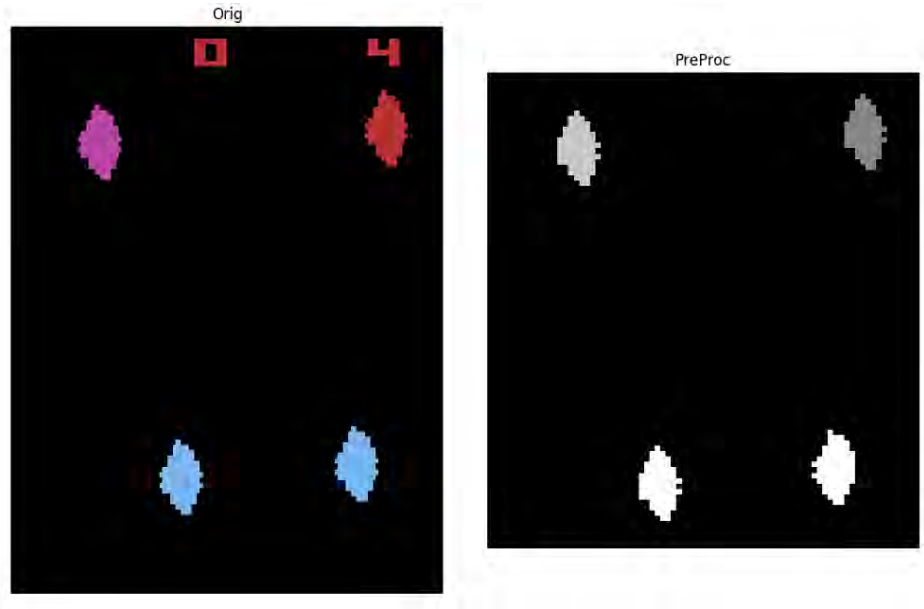


Figure 4.25. The initial frame of Asteroids, unprocessed and pre-processed.

The least effective game was Video Pinball, which is a simulation of a Pinball machine on the Atari. The most likely reason for the degrading performance of the network was due to the inherent randomness of the Pinball game. There is very little input that can affect the path of the ball as long as it is not near the bumpers. This does not happen very often during the game, and also provides no reward, so the network has no chance to learn this action and is ignored over randomly pressing buttons as the ball hits score giving objects.

Interestingly the network developed certain quirks in behaviour while training on this game, such as when the ball hit a certain object in the machine that made the screen flash white, the network would start hitting the flippers, even if the ball was nowhere near them. This was possibly due to a random relation being reinforced as the ball hit more objects as the flash occurred and flippers were being pressed at the time, leading to it becoming a learned behaviour. Unfortunately, this did not translate into a better overall score with more training.

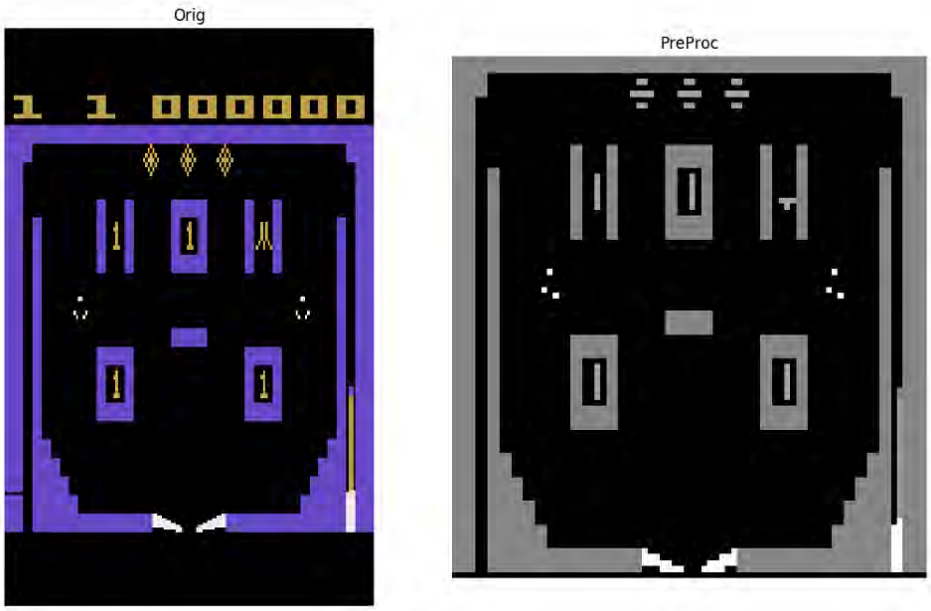


Figure 4.26. The initial frame of Pinball, unprocessed and pre-processed.

The last game and the most interesting one that deviated from this theory was Pong. Here, the neural network paddle on the right is competing with the game AI paddle on the left.

In contrast to the previous games, this breaks the trend of games where the action does not immediately correlate with a reward. Initially, it was assumed the game would do poorly and seeing the results of the game in negative also gave that impression. However, the negative score is a result of the OpenAI gym system taking the opponents scores away from the player's. This means that any score above 0 is actually a win for the network. While it never reached a full win, as seen in the graph in Figure 4.18, in testing the paddle successfully tracked and returned the ball and managed to score a significant amount of points against the game's AI.

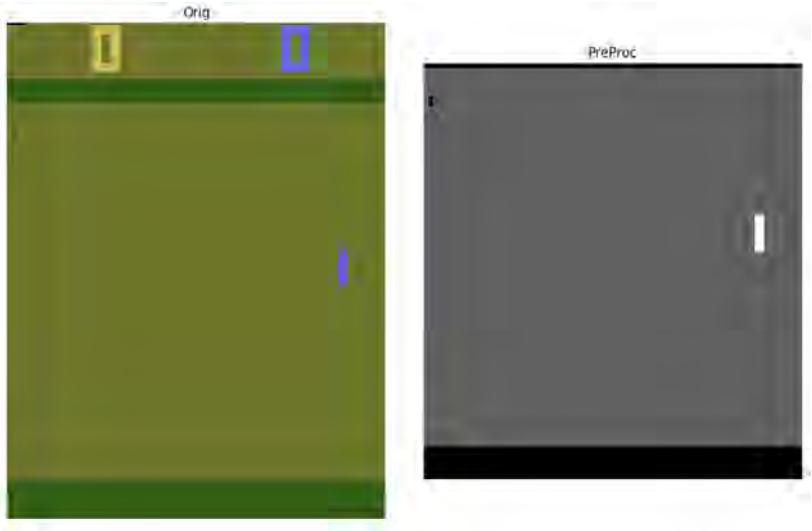


Figure 4.27. The initial frame of Pong, unprocessed and pre-processed.

4.11.4 Conclusion

All games except for Pong followed the hypothesis that the more closely a reward is tied to an action in the game, the better the network would be able to be trained on it. Qbert and Space Invaders showed the correlation with immediate reward actions, and Pinball and Asteroids showed it with delayed reward actions.

However, Pong did not fit the trend, due to it being able to improve via training, but having the score tied to scoring a goal, which is not a direct result of the player's action.

However, analysing the game further may give an indication as to what the network may be learning from. In the Pong game, the game AI's paddle follows the ball almost exactly at a consistent speed. The neural network paddle behaves erratically, but also follows the ball almost exactly, not leading it or predicting its position. This could mean that the network detected a correlation between the position of the opponents paddle instead, and mainly mimicking that actions of that paddle, due to it being able to score when closely following it, and therefore also hitting the ball.

While this cannot be directly confirmed, it is a likely explanation as to why Pong does not fit the trend of the rest of the games.

4.12 Summary

In this chapter, experiments were performed to evaluate the neural network and its use in playing a variety of games. In summary, these experiments highlighted some of the strengths and weaknesses in training a neural network to play video games, and also some challenges and techniques that can be used to overcome them.

Though the network was not able to play any game in these experiments to anything more than a novice level, that was not the main purpose of the project, and instead was used to highlight which techniques and strategies were best able to help train the AI, which will be discussed in detail in the next chapter.

The next chapter will discuss the overall results in detail and make comparisons between the different experiments.

5 Overall Results and Discussion

5.1 Introduction

Overall, these experiments were able to provide some of an answer towards the what the challenges and most effective strategies were for teaching machine learning AI to play video games. Overall the results from the experiments were very varied, especially with games other than Ms. Pac-Man

5.2 Challenges

One of the main challenges highlighted by these experiments was the need for a correlation between an action and an immediate reward.

As shown in Breakout, Pinball, and Asteroids, actions in these games do not immediately provide a reward. Perhaps more significantly, the actions that are required to play these games well (predicting the ball in Breakout, hitting the ball in Pinball, and avoiding rocks in Asteroids), do not reward any score. Instead, rewards are given based on actions that are somewhat out of the player's direct control, the most egregious example being Pinball, where very little intervention is required to achieve a high score.

In contrast to those, games where the action require to play the game effectively is both immediate and gives a reward, were able to be played best by the neural network. These were Ms. Pac-Man and Qbert, here Ms. Pac-Man eating pills, and Qbert jumping to blocks both are needed to properly play the game and are also tied with the movement of the character, leading to a strong correlation between action and reward. This means the network can appropriately learn correct actions due to the reward being given for actions that are required to play the game well.

The major exception to this theory is Pong, where the act of scoring a point is not directly tied to any particular action of the paddle, and instead requires the player to return the ball and fire it past the opponent. Seemingly this does not fit the trend of the other games, however, there is a potential explanation.

The network mainly learns by associating an action with a certain state in the game, meaning that it considers all details of the game being given as input. One aspect of the game besides the ball is the opposing game AI's paddle. This paddle's movement matches the ball almost exactly unless the ball is moving too fast. This means that

even when the ball is about to hit the opponents goal, it mimics the actions of the ball almost exactly.

This could mean that the network, instead of associating the position of the ball with the reward, associated the position of the opposing networks paddle. This means that when the goal was scored, and the network's paddle was in the same position as the AI's, it marked that as a positive action. This coincidentally was also the correct behaviour to at least play at a novice level against the AI, by mimicking it.

Therefore, Pong shows that unintended side effects that can occur when feeding data into a neural network, where the decisions it can make can be opaque to outside viewers, another challenge of attempting to train a neural network.

5.3 Strategies

Due to its use in AI research, and being the game most successfully trained, Ms. Pac-Man was the main game used for the experimentation of which strategies and techniques were most effective.

Overall for the Ms. Pac-Man series of experiments, very few techniques were able to improve on the Control experimental setup tested, with a few notable exceptions. The Reduce Large Rewards and No Large Rewards experiments performed better than the Control and also averted some of the negative behaviour exhibited by the Control experiment, mainly sticking to corners with power pellets.

This appeared to be mainly due to the large rewards linked to those corners, eating the power pellets and then the subsequent ghosts that ran into Ms. Pac-Man, which had much larger rewards than the rest of the scoring actions in the game. This shows at least, the requirement to make sure that rewards of a scenario are proportional to their "usefulness" to the AI.

Additional experiments concerning the rewards given to the AI for time spent alive, and negative rewards for dying, did not produce the effects hoped for and were potentially even degenerative to the network. The reward for time spent alive actually produced a perverse incentive for the AI to do nothing, as doing nothing gave it an increased score, which was reinforced by the increasing reward independent of any actions.

The negative reward was more difficult to explain but was most likely applied to the wrong frames. The frame just before or after death begin given a negative reward did not actually give the correlation the network needed to understand that the situation was undesirable. Even applied to the frame before death, it was unlikely the network would be able to do anything about it.

For the experiments with modifying network learning or changing input data, results were either worse than the Control, or inconclusive.

The layering approach used in both Ms. Pac-Man and Breakout produced an agent that was no better than random, which likely pointed to an error with the experiment, as all information was still available to the network, and it very little data was different between the Control experiments.

The use of colour in the experiment did not appear to have any major effect on the training of the network, and was likely superfluous, due to the nature of the neural network being able to differentiate between specific values of greyscale as easily as a red, green, and blue colour palette.

The introduction of more randomness to the experiment by use of a higher epsilon value slowed down training of the network, mostly due to making the network take less trained actions, and therefore slowing the learning of productive behaviours. This indicated that while some randomness appears to be needed for the AI to effectively learn new behaviours, there is a certain level where it instead hinders training.

5.4 Summary

This chapter discussed the results of the experiments and compared them as a whole. The major differenced were highlighted between the Ms. Pac-Man series of experiments, and those from other games, along with what particular changed made affected the overall results of each game environment.

The next chapter will draw conclusions from these results, and answer the aims, objectives, and academic question.

6 Conclusion

6.1 Aims and Objectives

From the literature review, a number of strategies were discussed around the teaching of machine learning AI's and their use in video games. These ranged from regular deep neural networks to convolutional neural networks, genetic algorithms, and separate modular networks with different hierarchies.

Apart from machine learning, the use of finite state machines or route trees was also brought up as a way of programming an AI to play video games, however, this was not the scope of the project, due to those system needing to be explicitly programmed.

Therefore, for this project, a convolutional deep neural network using Q reinforcement learning was chosen, mostly due to it fitting the frameworks in use such as OpenAI gym, and also due to existing research, such as a prominent research paper from DeepMind using a similar method to train an AI to play Ms. Pac-Man (Mnih et al., 2013).

The development of the network was done using TensorFlow, due to the flexibility and support shown from the literature review. The use of OpenAI gym followed on from this decision, as both used Python as their main scripting language. This then allowed other games to be implemented due to the Atari emulator in use.

The implementation of the network and its use for experimentation faced some challenges but were able to be overcome. For example, for quicker training times, GPU acceleration was added to the project, and a computer with a GPU and constant uptime was required. As this was resolved, a larger amount of experiments was able to be run in the limited time available.

6.2 Overall

From the overall experimentation, challenges from the experiments mainly surrounded the correlation between rewards and actions. Thus, to properly train an AI, it would be most productive to give appropriately scaled rewards to actions that are necessary for the AI to perform well. Examples of this would be giving a reward to the Breakout game every time it successfully hits the ball with its paddle, but not too high to incentivise that behaviour rather than hitting blocks. Another would be giving a negative reward

in Ms. Pac-Man the closer the player is to a ghost, enough to produce a behaviour that avoids even getting close to a game failure.

The other game experiments showed varying results, with some showing a weak, but positive increase in effectiveness, such as Space Invaders, with others showing no improvement, or negative effectiveness, such as Pinball and Asteroids. The main correlation between whether the network improved seemed to be the action to reward correlation discussed above, but it was also possible that certain aspects of the games made it more difficult for the network to learn correctly.

For example, in Space Invaders, the aliens speed up as more of them are destroyed, meaning that learned behaviours by the network, such as when to shoot, were often invalid, due to the bullet now going too slowly to hit an enemy. In the Asteroids game, a regular player would be able to hold both a directional button continually, and also fire, whereas in this project's network implementation, only one action could be passed to the game at a time.

Certain quirks of the emulator may also have contributed to this problem, such as random frame skipping making it difficult to get a clean set of movements from the game, and also due to the flickering of objects in the emulator, which was consistent with an actual Atari, but made evaluating game states harder, as objects that were required for the AI to evaluate were often not in the game information being returned to the network, such as ghosts not appearing in Ms. Pac-Man, and asteroids not showing up in Asteroids.

All these point to challenges of machine learning, partially due to the inability to tell exactly how a network has come to a decision, such as the strange quirk of the Pinball game hitting the paddles only when the screen was flashing, and partially due to the information being given to the network not being perfect all the time. While the latter aspect can be mitigated by instrumenting certain amounts of the data and passing it to the network, the former is more difficult to evaluate, and would likely require a deeper knowledge of neural networks than what was gleaned by the literature review and these experiments.

For other techniques, levelling out or reducing distracting rewards (power pellets and ghosts) also proved to be useful, as long as the goal of the network was not to achieve the highest score in the shortest time, but to perform a different or more long-term

goal, such as completing a level or surviving for the longest time. These experiments achieved higher results than the Control experiments, and also seemed to have more consistent games, and a steadier learning rate.

As evidenced by the Pong game, some indication of correct behaviour may also be sufficient to train an AI, rather than needing to program specific rewards. With the AI associating the opponent's paddle with a reward, it unintentionally learned to play the game by following the paddle, and by extension the ball. While this was unintentional, it led to an effective way to train the AI.

One aspect that was not as obvious in the experiments was that reducing the amount states the game was able to store as previous memory always resulted in a decreased in overall performance. This could only be tested downwards, due to hardware constraints, but given a larger amount of RAM, this aspect could also significantly improve the overall performance of the neural network.

Overall, the challenges and techniques were demonstrated on video games, but could also be applied to almost any reinforcement based neural network. Adjusting rewards is a large part of the process, and making sure those rewards are relevant, and that they are encouraging the correct behaviour without unintended side effects, is one of the main challenges. In addition, the challenge of not knowing exactly how the network came to a performing a specific action is one of the main limiting factors in determining effective techniques. However, as discovered through experimentation, via careful management of additional data and adjustment of rewards for the experiment, a more effective network was able to be trained.

7 Critical Evaluation and Processes

7.1 Overall

Overall, the general progress and results of the project reflected its initial goals. However, there were aspects that could have been improved or further developed, mainly concerning experiments and time management.

7.2 Design

7.2.1 Positives

The initial decision to make use of a machine learning and AI testing framework such as TensorFlow and OpenAI gym significantly sped up development time and further implementation when challenged appeared. Due to the community support available for both tools, and the further research that used them, they were both integral to the project's experiments.

The design of the neural network to be convolutional helped in the scope of the project, by allowing the network to play a number of other games through image analysis, rather than from being tied to any specific game.

7.2.2 Areas for Improvement

The initial design of the neural network was simplistic. For example, most modern implementations of a neural network reduce the learning rate over time to achieve better results (Zeiler, 2012), but in the network used for the experiments, the learning rate was kept static over time. While this was compensated for somewhat by the introduction of randomness controlled by the epsilon value, it could have sped up training times by starting with a high learning rate, and then adjusting it downwards.

The design of the network was also geared mainly towards image analysis, which while useful for the experiment on video games, may not be general enough to satisfactorily train other scenarios. While this was not an overt goal of the project, it would have been an interesting area of research to apply these techniques and learning to scenarios outside of gaming.

7.3 Implementation

7.3.1 Positives

GPU acceleration was integral to the project and the number of experiments could not have been carried out without it. Learning about GPU acceleration, as well as its

application in machine learning through further research was also a positive learning experience, as well as the practical benefit of faster training times.

7.3.2 Areas for Improvement

Many of the experiments required bespoke changes to be made to the script. Initially, these were easy to implement, but further along in the project, they became more difficult to fully integrate into the script without causing issues, and in one case potentially invalidating a large amount of experimental data. This could have been improved with a better experimental plan, but some of the issues could not have been foreseen, such as incompatibility with certain python module dependencies, so could only have been tackled by starting the implementation earlier.

7.4 Experimentation

7.4.1 Positives

A high number of experiments were performed, and a large amount of useful data was gathered through those experiments. These were made possible by procuring hardware capable of always running training, accelerating training, discussion of experiments with supervisors, and having an experimental plan early. This all led to data able to support a number of points that helped to answer the academic question.

7.4.2 Areas for Improvement

Many experiments that were planned were also not able to be done within the time constraints. A minor issue with this was the lack of space in the final report to highlight them, but otherwise, the experiments simply could not be run due to time constraints, despite the faster training procured in the project.

This could likely have been solved by starting the implementation process earlier, as the main lag time was the training time for each model the network had to train. With a quicker implementation, this would have left time during training to work on other aspects of the project, such as further research, or a more extensive experimental plan.

7.5 Testing and Errors

7.5.1 Positives

By running a control experiment for every scenario, a baseline of experimental performance could be established for each game and network model. This also gave

easy performance metrics to compare against, along with an initial dataset of random play which could verify if a network was actually learning, against just playing randomly.

7.5.2 Areas for Improvement

One aspect that could have been improved on the project was testing and validation. As the network often took a long time to run, and the results were often not visible until near the end of the running cycle, any errors made in the modification of the network would not be easily found or would be assumed to be a poor setup for training.

This occurred when adding functionality to the network to allow for different games and image processing to be used. The error was that the previous frame of the game was passed to the network, and then passed again as the next set of observations, essentially meaning that no change was visible to the AI. This was caught when re-running the control experiment and seeing no improvement in training, but in previous experiments was assumed to be from poor network parameters and techniques. This, in turn, cost the project several days of training time.

For future projects, running control experiment after major changes to the network could have been done to catch this error, but more general unit testing could also have been applied to the project to check the validity of separate functions in the script to verify that all components worked as they should.

More sanity testing would also have been useful, as partway through the project, it needed to be moved to new hardware. In that time, software tools used by the project had updated to new version and caused some deprecated functions to break. Establishing a proper installation plan and testing methodology would have lessened the impact of this change, and any potential future change due to hardware issues or otherwise.

7.6 Time Management and Planning

7.6.1 Positive

The production of an initial Gantt chart, as seen in Appendix A1, with clear deadline and time allocated allowed for a more managed approach to the project, with tasks clearly laid out and ordered in priority of completion. This gave an easy view of the status of the project and allowed all dependent tasks to be moved to their correct order,

which their dependencies were finished (i.e. completing the literature reviews, then the experiment plan, then performing the experiments).

7.6.2 Areas for Improvement

Time management was a generally difficult part of the project, with developmental time often competing with other aspects of the project, such as the write-up, literature review, and other external factors.

While time was estimated with these in mind with the initial Gantt chart, the estimates were made very generally, and often did not reflect real-life time taken. For example, while a large amount of time was given exclusively to training, this was able to be done in parallel with other tasks while the training was being run, the final Gantt chart as seen in Appendix A2 reflects how these tasks took up the majority of the time.

However, this was not effectively managed, with many other aspects of the project being given priority before training even started. This meant a lot of time spent training could have been used for other tasks, and if that had been completed sooner, would have allowed more experiments to be run.

For future improvement, identifying these "parallel" tasks would be most useful at the start of the project, and should be scheduled alongside other "serial" tasks instead of letting that time go to waste.

7.7 Research and Learning

7.7.1 Positives

From the initial proposal report to the literature view of the report, to the running of the experiments, research was a primary focus of this project, both in terms of learning and output.

The initial research that was done greatly benefitted the project and helped avoid certain problems, such as the techniques used in the structure of the project to avoid local minima. Other research gave direction for the experiments to do, such as changing the rewards, actions that gave rewards, the structure of the network, and additional information to pass through to the game.

They also helped give context the project, in terms of the application of deep learning techniques to areas outside of video games, and also helped set reasonable expectation of the performance of the network in terms of gameplay performance. It

also set expectations based on the hardware required for the project, and the computational power needed, which meant that experiments could be planned with more certainty.

7.7.2 Areas for Improvement

One main issue with the research step was the lack of understanding about the topics of machine learning and deep learning at the beginning of the project. This meant that many of the points brought up during research were not fully understood until experimentation was started and challenges were encountered.

This meant that some challenges that were actually addressed during research were also encountered through the project, without realising what the implications were. For example, part of the conclusion of networks not being able to train effectively with sparse rewards was brought up in a research paper about general deep learning playing video games (Justesen et al., 2017). Another issue was encountered with not being able to discern why a network was performing an action, such as the Pinball game hitting the paddles when the screen flashed. These were discussed in reviewed research, but the implications at the time were not clear.

Due to this, research would likely need to be a scheduled through all parts of the project, and reviews of existing knowledge should be done regularly, to ensure that any learned knowledge can be used to glean more relevant information from existing research.

7.8 Reflection

Overall the project goals were achieved, with the conclusions addressing the academic question, and the aims and objectives of the project achieved. However, there were secondary goals, mainly learning about deep learning, and about project management and development.

The main revelation during the project was the problem of initial lack of knowledge hindering understanding of research. During the literature review, a number of relevant topics and challenges were found in research, but due to lacking experience and understanding, they were unable to be fully understood, leading to the same knowledge having to be gleaned from experimental results and conclusions, such as the information about sparse rewards hindering learning, which was discovered independently.

Another example was the selection of a machine learning framework. While TensorFlow was selected, it was unknown at the time that certain features would be required such as GPU support, data visualisation, and certain neural network features such as convolutional layers. These were not known at the start of the experiment and were only discovered due to experience from testing the initial networks.

However, the research was also valuable in the beginning with setting project scope and expectations, especially about the level the AI could reasonably be trained to, and how a large amount of processing power was required. Also, revisiting the research gave more information due to the information gathered during experimentation, with previously confusing information being understood.

For future projects, project management could likely be improved by identifying parallel tasks earlier in the project, and allocation more estimated time to the implementation and design steps. These are the areas where issues were found the most and required the most amount of time to fix. This would also allow time for continuous research, to allow for improving experiments as more knowledge is gained, and being able to improve the project gradually, rather than in the predefined research times.

However, this does not mean that the initial research should be dropped, rather research should be integrated throughout the project, as new information is gathered, older research may take on a new relevance.

Overall, these lessons were a valuable experience that would not have otherwise been gained from doing a smaller project. The independence and wider scope allowed more challenges to occur, which could then be solved and learned from. These lessons can then be used to improve in any other project in the future, based on deep learning or otherwise.

References

Amazon Web Services (2018) TensorFlow on AWS - Deep Learning on the Cloud, *Amazon Web Services, Inc.*, [online] Available from: https://aws.amazon.com/tensorflow/ (Accessed 1 May 2018).

Auer, P., Burgsteiner, H. and Maass, W. (2008) A learning rule for very simple universal approximators consisting of a single layer of perceptrons, *Neural Networks*, **21**(5), pp. 786–795.

Bahrampour, S., Ramakrishnan, N., Schott, L. and Shah, M. (2015) Comparative Study of Deep Learning Software Frameworks, *arXiv:1511.06435 [cs]*, [online] Available from: http://arxiv.org/abs/1511.06435 (Accessed 12 April 2018).

Canonical Ltd. (2018) Ubuntu PC operating system | Ubuntu, *Ubuntu*, [online] Available from: https://www.ubuntu.com/desktop (Accessed 10 May 2018).

Chen, C., Seff, A., Kornhauser, A. and Xiao, J. (2015) DeepDriving: Learning Affordance for Direct Perception in Autonomous Driving, In IEEE, pp. 2722–2730, [online] Available from: http://ieeexplore.ieee.org/document/7410669/ (Accessed 5 May 2018).

Ciregan, D., Meier, U. and Schmidhuber, J. (2012) Multi-column deep neural networks for image classification, In *2012 IEEE Conference on Computer Vision and Pattern Recognition*, pp. 3642–3649.

Deng, L. and Yu, D. (2014) Deep Learning: Methods and Applications, *Foundations and Trends® in Signal Processing*, **7**(3–4), pp. 197–387.

GitHub (2018) A hackable text editor for the 21st Century, *Atom*, Atom, [online] Available from: https://atom.io/ (Accessed 10 May 2018).

Glorot, X. and Bengio, Y. (2010) Understanding the difficulty of training deep feedforward neural networks, In *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, pp. 249–256, [online] Available from: http://proceedings.mlr.press/v9/glorot10a.html (Accessed 4 May 2018).

Google (2018a) Compute Engine - IaaS, *Google Cloud*, [online] Available from: https://cloud.google.com/compute/ (Accessed 1 May 2018).

Google (2018b) Google Compute Engine Pricing | Compute Engine Documentation, *Google Cloud*, [online] Available from: https://cloud.google.com/compute/pricing (Accessed 1 May 2018).

Guzhva, A., Dolenko, S. and Persiantsev, I. (2009) Multifold Acceleration of Neural Network Computations Using GPU, In *Artificial Neural Networks – ICANN 2009*, *Lecture Notes in Computer Science*, Springer, Berlin, Heidelberg, pp. 373–380, [online] Available from: https://link.springer.com/chapter/10.1007/978-3-642-04274-4_39 (Accessed 11 April 2018).

Hagan, M. T., Demuth, H. B. and Beale, M. H. (1996) *Neural network design*, Pws Pub. Boston.

Handa, H. and Isozaki, M. (2008) Evolutionary fuzzy systems for generating better Ms.PacMan players, In *2008 IEEE International Conference on Fuzzy Systems (IEEE World Congress on Computational Intelligence)*, pp. 2182–2185.

Ioffe, S. and Szegedy, C. (2015) Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift, *arXiv:1502.03167 [cs]*, [online] Available from: http://arxiv.org/abs/1502.03167 (Accessed 10 May 2018).

Juliani, A. (2016) Simple Reinforcement Learning with Tensorflow Part 4: Deep Q-Networks and Beyond, *Medium*, [online] Available from: https://medium.com/@awjuliani/simple-reinforcement-learning-with-tensorflow-part-4-deep-q-networks-and-beyond-8438a3e2b8df (Accessed 15 April 2018).

Justesen, N., Bontrager, P., Togelius, J. and Risi, S. (2017) Deep Learning for Video Game Playing, *arXiv:1708.07902 [cs]*, [online] Available from: http://arxiv.org/abs/1708.07902 (Accessed 4 May 2018).

Kalyanpur, A. and Simon, M. (2001) Pacman using genetic algorithms and neural networks, *University of Maryland*.

Kawaguchi, K. (2016) Deep Learning without Poor Local Minima, In *Advances in Neural Information Processing Systems 29*, Lee, D. D., Sugiyama, M., Luxburg, U. V., Guyon, I., and Garnett, R. (eds.), Curran Associates, Inc., pp. 586–594, [online] Available from: http://papers.nips.cc/paper/6112-deep-learning-without-poor-local-minima.pdf.

Keras (2018) Keras Documentation, *Keras Documentation*, [online] Available from: https://keras.io/ (Accessed 4 May 2018).

Korolev, S. (2017) Neural Network to play a snake game, *Towards Data Science*, [online] Available from: https://towardsdatascience.com/today-im-going-to-talk-about-a-small-practical-example-of-using-neural-networks-training-one-to-6b2cbd6efdb3 (Accessed 1 May 2018).

Kovalev, V., Kalinovsky, A. and Kovalev, S. (2016) Deep Learning with Theano, Torch, Caffe, Tensorflow, and Deeplearning4J: Which One is the Best in Speed and Accuracy?, [online] Available from: http://elib.bsu.by/handle/123456789/158561 (Accessed 1 May 2018).

LeCun, Y., Bengio, Y. and Hinton, G. (2015) Deep learning, *Nature*, **521**(7553), pp. 436–444.

Li, X. and Wu, X. (2014) Constructing Long Short-Term Memory based Deep Recurrent Neural Networks for Large Vocabulary Speech Recognition, *arXiv:1410.4281 [cs]*, [online] Available from: http://arxiv.org/abs/1410.4281 (Accessed 14 April 2018).

Lucas, S. M. (2005) Evolving a Neural Network Location Evaluator to Play Ms. Pac-Man, In *Proceedings of the 2005 IEEE Symposium on Computational Intelligence and Games (CIG 2005)*, Kendall, G. and Lucas, S. (eds.), IEEE, pp. 203–210, [online] Available from: http://cswww.essex.ac.uk/cig/2005/papers/p1058.pdf.

Luke, S., Cioffi-Revilla, C., Panait, L., Sullivan, K. and Balan, G. (2005) MASON: A Multiagent Simulation Environment, *SIMULATION*, **81**(7), pp. 517–527.

Matplotlib (2018) Matplotlib: Python plotting — Matplotlib 2.2.2 documentation, *Matplotlib*, [online] Available from: https://matplotlib.org/ (Accessed 10 May 2018).

Milutinovic, M., Baydin, A. G., Zinkov, R., Harvey, W., Song, D., Wood, F. and Shen, W. (2017) End-to-end Training of Differentiable Pipelines Across Machine Learning Frameworks, [online] Available from: https://openreview.net/forum?id=ryh7qqGRZ&noteId=ryh7qqGRZ (Accessed 1 May 2018).

Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D. and Riedmiller, M. (2013) Playing Atari with Deep Reinforcement Learning, *arXiv:1312.5602 [cs]*, [online] Available from: http://arxiv.org/abs/1312.5602 (Accessed 15 April 2018).

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S. and Hassabis, D. (2015) Human-level control through deep reinforcement learning, *Nature*, **518**(7540), pp. 529–533.

Nielsen, M. A. (2015) Neural Networks and Deep Learning, [online] Available from: http://neuralnetworksanddeeplearning.com (Accessed 12 April 2018).

NumPy (2018) NumPy — NumPy, *NumPy*, [online] Available from: http://www.numpy.org/ (Accessed 10 May 2018).

Oh, J., Guo, X., Lee, H., Lewis, R. L. and Singh, S. (2015) Action-Conditional Video Prediction using Deep Networks in Atari Games, In *Advances in Neural Information Processing Systems 28*, Cortes, C., Lawrence, N. D., Lee, D. D., Sugiyama, M., and Garnett, R. (eds.), Curran Associates, Inc., pp. 2863–2871, [online] Available from: http://papers.nips.cc/paper/5859-action-conditional-video-prediction-using-deep-networks-in-atari-games.pdf (Accessed 10 May 2018).

OpenAI (2018) Gym: A toolkit for developing and comparing reinforcement learning algorithms, [online] Available from: https://gym.openai.com (Accessed 12 April 2018).

Parthasarathy, D. (2016) Write an AI to win at Pong from scratch with Reinforcement Learning, *Medium*, [online] Available from: https://medium.com/@dhruvp/how-to-write-a-neural-network-to-play-pong-from-scratch-956b57d4f6e0 (Accessed 1 May 2018).

Parvat, A., Chavan, J., Kadam, S., Dev, S. and Pathak, V. (2017) A survey of deep-learning frameworks, In *Inventive Systems and Control (ICISC), 2017 International Conference on*, IEEE, pp. 1–7, [online] Available from: https://ieeexplore.ieee.org/document/8068684/.

Python Software Foundation (2018) Welcome to Python.org, *Python.org*, [online] Available from: https://www.python.org/ (Accessed 10 May 2018).

Robles, D. and Lucas, S. M. (2009) A simple tree search method for playing Ms. Pac-Man, In *Proceedings of the 5th international conference on Computational Intelligence and Games*, Milano, Italy, IEEE Press, pp. 249–255, [online] Available from: https://ieeexplore.ieee.org/document/5286469/.

Samuel, A. L. (1959) Some Studies in Machine Learning Using the Game of Checkers, *IBM Journal of Research and Development*, **3**(3), pp. 210–229.

Schmidhuber, J. (2015) Deep learning in neural networks: An overview, *Neural Networks*, **61**, pp. 85–117.

Schrum, J. and Miikkulainen, R. (2014) Evolving multimodal behavior with modular neural networks in Ms. Pac-Man, In ACM Press, pp. 325–332, [online] Available from: http://dl.acm.org/citation.cfm?doid=2576768.2598234 (Accessed 11 April 2018).

van Seijen, H., Fatemi, M., Romoff, J., Laroche, R., Barnes, T. and Tsang, J. (2017) Hybrid Reward Architecture for Reinforcement Learning, *arXiv:1706.04208 [cs]*, [online] Available from: http://arxiv.org/abs/1706.04208 (Accessed 11 April 2018).

Spencer-Harper, M. (2015) How to build a simple neural network in 9 lines of Python code, *Medium*, [online] Available from: https://medium.com/technology-invention-and-more/how-to-build-a-simple-neural-network-in-9-lines-of-python-code-cc8f23647ca1 (Accessed 11 April 2018).

Stone, P. and Veloso, M. (2000) Multiagent Systems: A Survey from a Machine Learning Perspective, *Autonomous Robots*, **8**(3), pp. 345–383.

TensorFlow (2018a) Installing TensorFlow for Java, *TensorFlow*, [online] Available from: https://www.tensorflow.org/install/install_java (Accessed 12 April 2018).

TensorFlow (2018b) Performance Guide, *TensorFlow*, [online] Available from: https://www.tensorflow.org/performance/performance_guide (Accessed 15 April 2018).

TensorFlow (2018c) *skflow: Simplified interface for TensorFlow (mimicking Scikit Learn) for Deep Learning*, Python, tensorflow, [online] Available from: https://github.com/tensorflow/skflow (Accessed 12 April 2018).

Veen, F. van (2016) The Neural Network Zoo, *The Asimov Institute*, [online] Available from: http://www.asimovinstitute.org/neural-network-zoo/ (Accessed 14 April 2018).

Vishnu, A., Siegel, C. and Daily, J. (2016) Distributed TensorFlow with MPI, *arXiv:1603.02339 [cs]*, [online] Available from: http://arxiv.org/abs/1603.02339 (Accessed 1 May 2018).

Wiering, M. (2000) Multi-Agent Reinforcement Leraning for Traffic Light Control, In *Proceedings of the Seventeenth International Conference on Machine Learning*, *ICML '00*, San Francisco, CA, USA, Morgan Kaufmann Publishers Inc., pp. 1151–1158, [online] Available from: http://dl.acm.org/citation.cfm?id=645529.658109 (Accessed 10 May 2018).

Williams, P. R., Perez-Liebana, D. and Lucas, S. M. (2016) Ms. Pac-Man Versus Ghost Team CIG 2016 Competition, *arXiv:1609.02316 [cs]*, [online] Available from: http://arxiv.org/abs/1609.02316 (Accessed 1 May 2018).

Zacharias, J., Barz, M. and Sonntag, D. (2018) A Survey on Deep Learning Toolkits and Libraries for Intelligent User Interfaces, *arXiv:1803.04818 [cs]*, [online] Available from: http://arxiv.org/abs/1803.04818 (Accessed 4 May 2018).

Zeiler, M. D. (2012) ADADELTA: An Adaptive Learning Rate Method, [online] Available from: https://arxiv.org/abs/1212.5701 (Accessed 16 April 2018).

Zhang, Q., Cheng, L. and Boutaba, R. (2010) Cloud computing: state-of-the-art and research challenges, *Journal of Internet Services and Applications*, **1**(1), pp. 7–18.

Bibliography

Abadi, M., Barham, P., Chen, J., Chen, Z., Davis, A., Dean, J., Devin, M., Ghemawat, S., Irving, G., Isard, M., Kudlur, M., Levenberg, J., Monga, R., Moore, S., Murray, D. G., Steiner, B., Tucker, P., Vasudevan, V., Warden, P., Wicke, M., Yu, Y. and Zheng, X. (2016) TensorFlow: A System for Large-scale Machine Learning, In *Proceedings of the 12th USENIX Conference on Operating Systems Design and Implementation*, *OSDI'16*, Berkeley, CA, USA, USENIX Association, pp. 265–283, [online] Available from: http://dl.acm.org/citation.cfm?id=3026877.3026899 (Accessed 11 May 2018).

Alpaydin, E. (2004) *Introduction to Machine Learning*, MIT Press.

Gallagher, M. and Ledwich, M. (2007) Evolving Pac-Man Players: Can We Learn from Raw Input?, In *2007 IEEE Symposium on Computational Intelligence and Games*, pp. 282–287.

Gill, N. S. (n.d.) Artificial Neural Networks & It's Applications - XenonStack Blog, *XenonStack - A Stack Innovator*, [online] Available from: https://www.xenonstack.com/blog/data-science/overview-of-artificial-neural-networks-and-its-applications (Accessed 14 April 2018).

Goldberg, D. E. and Holland, J. H. (1988) Genetic Algorithms and Machine Learning, *Machine Learning*, **3**(2–3), pp. 95–99.

Guo, X., Singh, S., Lee, H., Lewis, R. L. and Wang, X. (2014) Deep Learning for Real-Time Atari Game Play Using Offline Monte-Carlo Tree Search Planning, In *Advances in Neural Information Processing Systems 27*, Ghahramani, Z., Welling, M., Cortes, C., Lawrence, N. D., and Weinberger, K. Q. (eds.), Curran Associates, Inc., pp. 3338–3346, [online] Available from: http://papers.nips.cc/paper/5421-deep-learning-for-real-time-atari-game-play-using-offline-monte-carlo-tree-search-planning.pdf (Accessed 11 May 2018).

Haykin, S. (1994) *Neural Networks: A Comprehensive Foundation*, 1st ed, Upper Saddle River, NJ, USA, Prentice Hall PTR.

Kingma, D. P. and Ba, J. (2014) Adam: A Method for Stochastic Optimization, *arXiv:1412.6980 [cs]*, [online] Available from: http://arxiv.org/abs/1412.6980 (Accessed 11 May 2018).

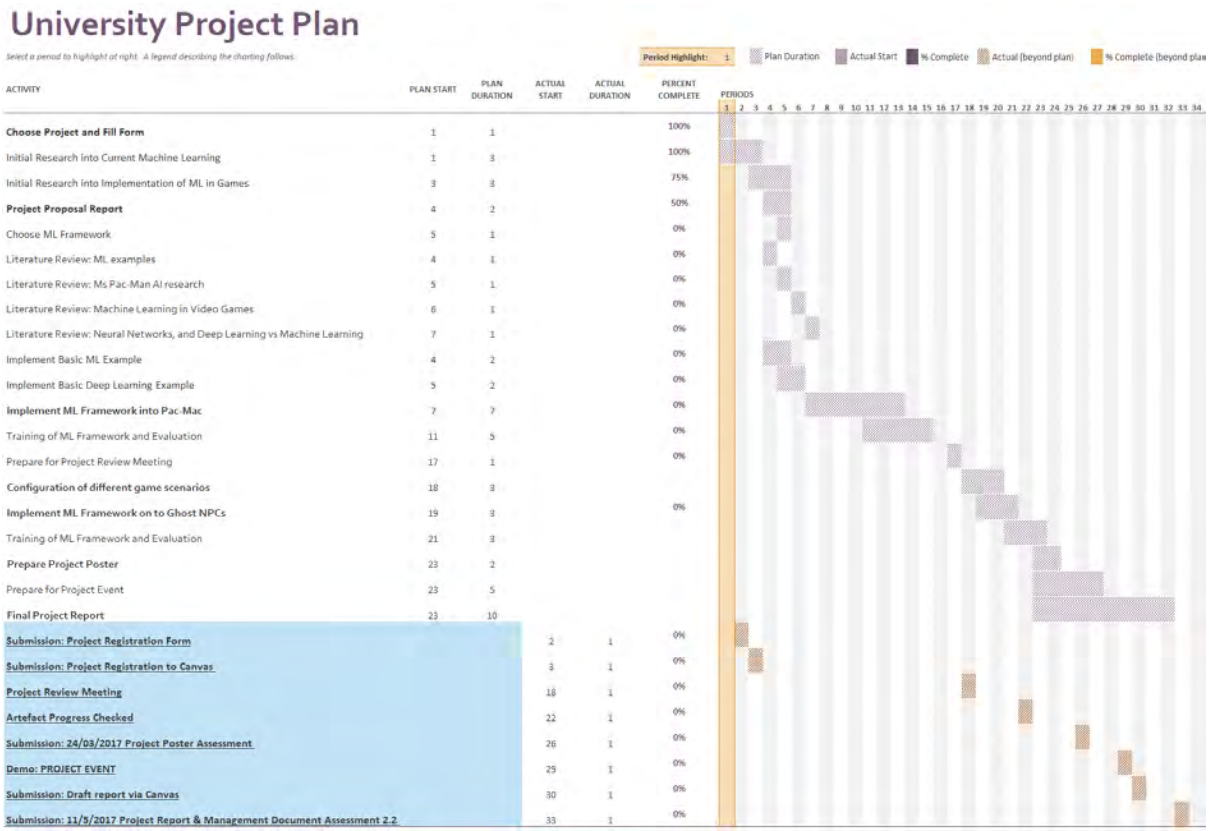
Watkins, C. J. C. H. and Dayan, P. (1992) Q-learning, *Machine Learning*, **8**(3–4), pp. 279–292.

Wittkamp, M., Barone, L. and Hingston, P. (2008) Using NEAT for continuous adaptation and teamwork formation in Pacman, In *2008 IEEE Symposium On Computational Intelligence and Games*, pp. 234–242.

Appendices

Appendix A1 - Initial Gantt Chart

Below is a screenshot of the project plan.



University Project Plan

Select a period to highlight at right. A legend describing the charting follows.

Period Highlight: 1 | Plan Duration | Actual Start | % Complete | Actual (beyond plan) | % Complete (beyond plan)

ACTIVITY	PLAN START	PLAN DURATION	ACTUAL START	ACTUAL DURATION	PERCENT COMPLETE
Choose Project and Fill Form	1	1			100%
Initial Research into Current Machine Learning	1	3			100%
Initial Research into Implementation of ML in Games	3	3			75%
Project Proposal Report	4	2			50%
Choose ML Framework	5	1			0%
Literature Review: ML examples	4	1			0%
Literature Review: Ms Pac-Man AI research	5	1			0%
Literature Review: Machine Learning in Video Games	6	1			0%
Literature Review: Neural Networks, and Deep Learning vs Machine Learning	7	1			0%
Implement Basic ML Example	4	2			0%
Implement Basic Deep Learning Example	5	2			0%
Implement ML Framework into Pac-Mac	7	7			0%
Training of ML Framework and Evaluation	11	5			0%
Prepare for Project Review Meeting	17	1			0%
Configuration of different game scenarios	18	3			
Implement ML Framework on to Ghost NPCs	19	3			0%
Training of ML Framework and Evaluation	21	3			
Prepare Project Poster	23	2			
Prepare for Project Event	23	5			
Final Project Report	23	10			
Submission: Project Registration Form			2	1	0%
Submission: Project Registration to Canvas			3	1	0%
Project Review Meeting			18	1	0%
Artefact Progress Checked			22	1	0%
Submission: 24/03/2017 Project Poster Assessment			26	1	0%
Demo: PROJECT EVENT			29	1	0%
Submission: Draft report via Canvas			30	1	0%
Submission: 11/5/2017 Project Report & Management Document Assessment 2.2			33	1	0%

The Gantt chart shows estimated times for various tasks through the project, and the status of some tasks which have already been started.

Bolded are some interim deliverables and major milestones. Major deliverables include:

- Project Proposal Form to be completed in week 1 which lists the basic ideas of the project, the academic question, and the three artefacts, along with any potential ethical considerations.
- Project Proposal Report, which goes into more detail about the project, including aim, objectives, and detailed descriptions of artefacts and plans.
- The implementation of the machine learning framework and the Pac-man game, which is the first-level artefact for the project, and where evaluations can begin.

- Implementation of the framework onto other games, which is the final level artefact, and could be the most complex aspect of the project
- Project poster, which is a summary of the entire project, and required for the demonstration event
- Final Project Report, which includes all details about the project, a literature review, findings and observations, and final conclusions

Major milestones include:

- Project proposal form hand-in which is the stage at which the project should be decided.
- Project proposal report hand-in where a more detailed picture of the project should be completed, and an initial view of the research.
- Project review meeting, where the progress of the project will be checked, and there will be a meeting with the supervisor and reader.
- Poster hand-in, which is required for the demonstration event.
- Demonstration event, where the project will be shown live to students, supervisor, and reader
- Draft hand-in for Final Project Report
- Final Project Report and Artefact hand-in deadline, after which the project will be completed.

A more detailed rundown of the tasks of the project plan are:

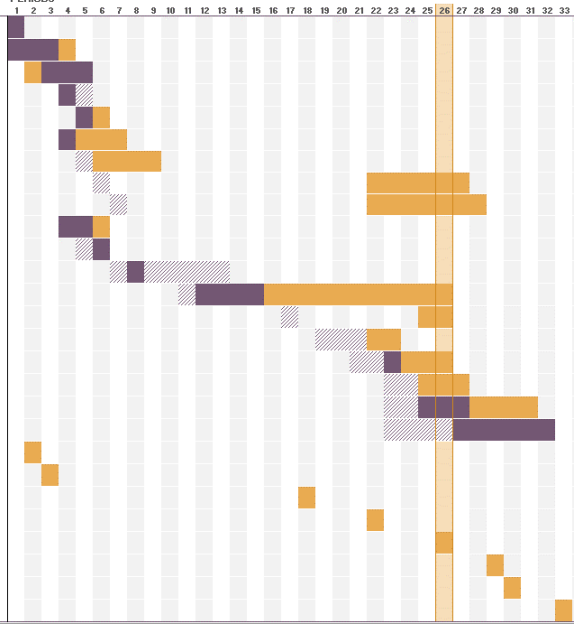
- Project proposal form. This task involves filling out the form, showing it to the supervisor, and getting it approved by the ethical board and signed.
- Initial Research into Current Machine Learning. Research into the current methodologies of machine learning.
- Initial Research into Implementation of ML in Games. Research into the ways and tools used to implement machine learning AI into video games.
- Project Proposal Report. Writing and complete the project proposal report, with aims, objectives, and more detailed descriptions and research.
- Choose ML Framework. The major open machine learning frameworks are TensorFlow and scikit-learn, and one will be chosen to proceed with the rest of the project.

- Literature Review. This is a review of current literature to assess current techniques, methods, and tools that have already been used and information that has already been researched. This is to avoid duplication of effort and avoid repeating already fully-assessed research.
- Implement Basic ML Example. To test the framework and to gather technical knowledge, a basic example needs to be done with the framework, to show machine learning capabilities.
- Implement Basic Deep Learning Example. As the topic is to do with deep learning, an example of the differences between simple machine learning and deep learning would be required to proceed.
- Implement ML Framework into Pac-Mac. The base artefact would be developed in this task, with knowledge from research and implemented examples being used to create a model for the game.
- Training of ML Framework and Evaluation. The project is about exploring strategies, so testing will need to be done by having the AI play the game, learning from the data, and then repeating this over a certain timescale, and across different methodologies. These can then be evaluated once data has been collected.
- Prepare for Project Review Meeting. Project review meeting would be the first meeting with the reader, so preparing notes and the artefact for demonstration would be done for the meeting.
- Implement ML Framework with other games. This would be modifying the framework to allow for the network to control other games. This would be the final level artefact task.
- Training of ML Framework and Evaluation. This is the second machine learning iteration for the evaluation of the framework on different games.
- Prepare Project Poster. The poster for the project would need to be created for the demonstration event.
- Prepare for Project Event. Preparing for the project event would be taking videos and preparing notes and the artefact for demonstrating at the event.
- Final Project Report. Completing the final project report would be collating all data and conclusions and typing the final dissertation.

Appendix A2 - Final Gantt Chart

University Project Plan

Select a period to highlight at right. A legend describing the charting follows.

Period Highlig 26 | Plan Duration | Actual Start | % Complete | Actual (beyond plan) | % Complete (beyond plan

ACTIVITY	PLAN START	PLAN DURATION	ACTUAL START	ACTUAL DURATION	PERCENT COMPLETE
Choose Project and Fill Form	1	1	1	1	100%
Initial Research into Current Machine Learning	1	3	1	4	100%
Initial Research into Implementation of ML in Games	3	3	2	4	100%
Project Proposal Report	4	2	4	1	100%
Choose ML Framework	5	1	5	2	100%
Literature Review: ML examples	4	1	4	4	100%
Literature Review: Ms Pac-Man AI research	5	1	6	4	100%
Literature Review: Machine Learning in Video Games	6	1	22	6	100%
Literature Review: Neural Networks, Deep Learning, and Machine Learning	7	1	22	7	100%
Implement Basic ML Example	4	2	4	3	100%
Implement Basic Deep Learning Example	5	2	6	1	100%
Implement ML Framework into Pac-Mac	7	7	8	1	100%
Training of ML Framework and Evaluation	11	5	12	15	100%
Prepare for Project Review Meeting	17	1	25	2	100%
Implement ML Framework with other games	19	3	22	2	100%
Training of ML Framework and Evaluation	21	3	23	4	100%
Prepare Project Poster	23	2	25	3	100%
Prepare for Project Event	23	5	25	7	100%
Final Project Report	23	10	27	6	100%
Submission: Project Registration Form			2	1	100%
Submission: Project Registration to Canvas			3	1	100%
Project Review Meeting			18	1	100%
Artefact Progress Checked			22	1	100%
Submission: 24/03/2017 Project Poster Assessment			26	1	100%
Demo: PROJECT EVENT			29	1	100%
Submission: Draft report via Canvas			30	1	100%
Submission: 11/5/2017 Project Report & Management Document Assessment 2.2			33	1	100%